UNIVERSIDAD ALFONSO X EL SABIO

ESCUELA BUSINESS & TECH

GRADO EN INGENIERÍA INFORMÁTICA

TRABAJO DE FIN DE GRADO

Del Bit al Comportamiento: Redefiniendo la Frontera de la Seguridad Mediante el Aprendizaje Profundo

Rubén Nogueras González

Junio de 2026

UNIVERSIDAD ALFONSO X EL SABIO
ESCUELA BUSINESS & TECH
GRADO EN INGENIERÍA INFORMÁTICA

DEL BIT AL COMPORTAMIENTO: REDEFINIENDO LA FRONTERA DE LA SEGURIDAD MEDIANTE EL APRENDIZAJE PROFUNDO

ALUMNO: RUBÉN NOGUERAS GONZÁLEZ
N.P.: 139174
CÓDIGO TFG: GMAI-25076
TUTOR ACADÉMICO: SARA IGNACIO CERRATO
FECHA DE PRESENTACIÓN:
BREVE DESCRIPCIÓN:

AÑADIR ESTO CUANDO ACABE EL TFG!!!

FIRMA DEL DIRECTOR DEL TRABAJO: **FIRMA DEL ALUMNO:**

Índice

Índice de figuras **3**

1. Introducción **5**

2. Estado del Arte **6**

3. Marco Teórico **7**
3.1. Detección de Anomalías 7
3.2. Autoencoder 7
3.3. Autoencoder Subcompleto 8
3.4. Random Forest 9
3.4.1. El estimador y el problema de la varianza 9
3.4.2. Bootstrap Aggregating (Bagging) 10
3.4.3. Feature Selection y Estimación Final 10
3.5. La Impureza de Gini en un Random Forest 11

4. Implementación **13**
4.1. Análisis Exploratorio de Datos EDA 13
4.2. Preprocesamiento 21
4.3. Autoencoder 25
4.4. Servidor Web Apache 28
4.5. Pruebas de Penetración con SQLMap e Hydra 30
4.6. Determinación del Umbral de Detección de Anomalias 33
4.6.1. Método de la Desviación Estándar 34
4.6.2. Método de la Percentiles 34
4.6.3. Método del Valor Máximo 35
4.7. Modelo de Clasificación 35
4.7.1. Importancia de Gini para la selección de Variables 36
4.7.2. Primeros resultados 40
4.7.3. Captura de datos personalizada 42
4.7.4. Resultados finales 43
4.8. Implementación de Notificaciones y Respuesta ante Incidentes vía Telegram API 46

5. conclusiones **48**

Bibliografía **50**

Índice de figuras

1. Estructura elemental de un Autoencoder. Obtenida a partir de (1) 8
2. Estructura del Dataset CICIDS2017. Datos disponibles en (2) 14
3. Distribución de datos benignos y maliciosos 15
4. Distribución de clases: Datos benignos y maliciosos 15
5. Distribución de datos maliciosos 16
6. Distribución de clases: Datos maliciosos 16
7. Dataframe de outliers 18
8. Histograma, boxplot y Q-Q plot sobre la variable 'Flow Duration' 18
9. Matriz de correlación inicial de los datos 20
10. Matriz de correlación preprocesada de los datos 22
11. Matriz de correlación final 24
12. Función ReLu y Lineal 27
13. Comparación de MSE para datos benignos y maliciosos. 27
14. Captura de flujo malicioso en tiempo real causado por hping3. 28
15. Pagina personalizada utilizada para el servidor web con Apache 29
16. Captura de flujo malicioso en tiempo real causado por SQLMap 31
17. Captura de flujo malicioso en tiempo real causado por Hydra 32
18. Captura de flujo malicioso en tiempo real a través de Python 33
19. Importancia de Gini sin procesamiento previo: Gráfico de barras 37
20. Importancia de Gini sin procesamiento previo: Dataframe 38
21. Importancia de Gini con datos procesados: Gráfico de barras 39
22. Importancia de Gini con datos procesados: Dataframe 40
23. Distribución de clases de carácter malicioso tras análisis de duplicados 41
24. Predicciones erróneas en la clasificación inicial 42
25. Distribución de clases de carácter malicioso del tráfico capturado 43
26. Muestras de datos capturados: SQL Injection 44
27. Metricas del entrenamiento del modelo final: RandomForest 45
28. Prediciendo ataque malicioso creado con SQLMap 45
29. Implementación con la API de Telegram 47

CAPÍTULO

1

Introducción

CAPÍTULO

2

Estado del Arte

CAPÍTULO

3

Marco Teórico

3.1. Detección de Anomalías

En esta sección vamos a explicar una parte vital de este proyecto, la cual consiste en la **detección de anomalías**. En el contexto de nuestro servidor web, una anomalía es un comportamiento extraño en el tráfico de red del servidor, sobre el cual nos vamos a centrar en aquellas peticiones de red fuera de lo común que se envían al servidor.

Más adelante vamos a saber diferenciar el tipo de ciberataque que se está intentando realizar por parte del atacante, pero en esta primera parte de la detección tenemos que diferenciar peticiones de red con un comportamiento benigno de uno maligno, y para ello vamos a utilizar un **autoencoder**.

3.2. Autoencoder

Un **autoencoder** es una red neuronal que intenta replicar los datos recibidos de su capa de entrada x en la capa de salida r. No obstante, en esta copia no nos vamos a fijar al utilizar cualquier *autoencoder* en general, ya que nos interesa conocer el **error** que la red comete al intentar replicar los datos en la capa de salida r. De esta manera, si obtenemos un valor de error alto, los datos introducidos no se asemeja a los datos de entrenamiento de la red, y si obtenemos un valor de error bajo, lo contrario. Precisamente esto es en lo que nos vamos a basar para detectar logs malignos en el servidor, ya que si entrenamos la red con un set de datos benignos y comparamos el error producido por otras entradas benignas y malignas, el error será mucho mayor en los conjuntos de datos que el modelo no conoce, es decir, los datos malignos.

Según se especifica en (1), este tipo de redes se componen en dos partes: El primer componente se denomina **encoder** f (transforma lo datos de entrada x en h), el cual identifica las variables más significativas y descarta el ruido innecesario, y un **decoder** g, que obtiene la reconstrucción del modelo r a partir de h. Esta es la estructura básica de la red y se puede ver reflejada en la siguiente imagen:

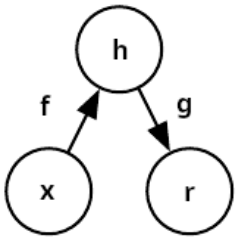

Figura 1: Estructura elemental de un Autoencoder. Obtenida a partir de (1)

Donde las funciones f y g serán definidas en la sección de implementación (3.2). Una vez se ha visto la estructura básica de un *autoencoder*, nos vamos a adentrar más específicamente en la red que se va a implementrar de manera práctica en este proyecto.

3.3. Autoencoder Subcompleto

Se ha visto en la sección anterior que nuestro modelo tiene que aprender de sus datos de entrenamiento, y esto se puede lograr con un **autoencoder subcompleto**. La diferencia con respecto a un autoencoder genérico es que se van a limitar los datos obtenidos en h, obligando a que tengan una dimensión inferior que x. De esta manera h es forzada a aprender las propiedades más representativas de los datos de entrenamiento, lo cual resultará en una reconstrucción final r con ciertas discrepancias con respecto a los datos de entrada x.

Podemos medir dichas discrepancias minimizando una función, la función de pérdida del modelo:

$$L(x, g(f(x))) \tag{3.1}$$

Donde L es la función de pérdida que identifica si la reconstrucción del modelo $r = g(f(x))$ obtiene resultados similares a x. El entrenamiento de la red se basa en minimizar la función de pérdida que acabamos de ver (3.1), y la función de pérdida que se va a usar en este trabajo es el **error cuadrático medio** *MSE*, ya que penaliza los errores en mayor medida al elevar al cuadrado, es fácil de optimizar al ser continua y derivable en todo punto del espacio (esto

facilita el entrenamiento de la red), representa la distancia euclídea por lo que facilita la representación, etc.

$$MSE = \frac{1}{n}\sum_{i=0}^{n}(x_i - r_i)^2 \tag{3.2}$$

Siendo x los datos de entrada, r los valores de la reconstrucción, y n el número total de datos en la capa de entrada y salida del modelo.

3.4. Random Forest

A continuación se va a ver detenidamente el funcionamiento teórico del modelo de clasificación utilizado en la implementación de este trabajo, el Random Forest. Este modelo fue escogido favorablemente en base a una comparación de resultados con otros modelos propuestos, debido a su mejor desempeño. Este apartado presenta un enfoque teórico - práctico, volcando la teoría a la práctica y su implementación en código.

3.4.1. El estimador y el problema de la varianza

Este modelo se compone de un boque de árboles, es decir, un conjunto de árboles de decisión. Un árbol de decisión es un modelo no paramétricos que segmenta el espacio de características mediante una serie de reglas que se van propagando hasta la decisión final. Por ello, destacan por su gran interpretabilidad y capacidad de capturar relaciones complejas entre las distintas variables de los datos, pero presentan un grave problema, la alta varianza.

Imaginemos que un árbol de decisión contiene un sesgo (*bias*) reducido ya que es capaz de adaptarse con mucha precisión a la estructura del conjunto de datos de su entrenamiento. Esta sesgo reducido introduce una gran flexibilidad que lo convierte en un estimador inestable, ya que cualquier pequeña fluctuación en los datos de entrenamiento puede suponer un drástico cambio en la estructura de ramas y nodos del propio árbol, y esto es lo que se conoce como alta varianza. Entonces, si se tiene un conjunto de datos de entrenamiento $\mathbf{Z} = \{(x_1, y_1), \ldots, (x_N, y_N)\}$, el algoritmo busca minimizar la impureza en cada partición, lo que suele conducir al sobreajuste ($overfitting$), comprometiendo la capacidad del modelo para generalizar datos distintos a los de su entrenamiento, ya que se memorizan dichos datos, perdiendo por completo la capacidad de predeicr correctamente cualquier otro dato.

3.4.2. Bootstrap Aggregating (Bagging)

Para aumentar la robustez de los árboles sin incrementar el sesgo, se emplea un método conocido como Bagging (*Bootstrap Aggregating*), el cual se basa en que el promedio de múltiples estimadores independientes reduce la varianza global en el sistema de predicción. Si se dispusiera de B conjuntos de entrenamiento independientes en un escenario ideal, cada uno con una varianza σ^2, la varianza del promedio sería σ^2/B. Sin embargo, en la práctica es imposible obtener múltiples conjuntos independientes, por lo que el Bagging simula esta condición mediante otro proceso conocido como **Bootstrap**.

Esta técnica consiste en generar B muestras distintas $\mathbf{Z}^{*b}$ mediante la extracción aleatoria de N instancias del conjunto original de árboles $\mathbf{Z}$, utilizando un muestreo con reemplazo. Esto produce que cada muestra de Bootstrap (cada árbol de nuestro modelo) contenga aproximadamente dos tercios de los datos originales, mientras que la cantidad restante se denominan datos *Out-of-Bag* (OOB), los cuales son fundamentales para la validación interna del modelo.

Tras la fase de entrenamiento de cada árbol T_b de cada muestra $\mathbf{Z}^{*b}$, la predicción final del modelo se obtiene comparando los resultados obtenidos de cada árbol. En un problema de clasificación como en nuestro caso, esto se traduce en el voto mayoritario de cada árbol:

$$\hat{f}_{bag}(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x) \tag{3.3}$$

Donde $\hat{f}_{bag}(x)$ representa la predicción, B el número total de estimadores (árboles) y $T_b(x)$ la salida de cada árbol independiente. De esta manera se reducen los errores de árboles sobreajustados, propiciando en un modelo significativamente más robusto frente al ruido inherente al tráfico de red.

3.4.3. Feature Selection y Estimación Final

La principal diferencia entre un *Random Forest* con respecto a un *Bagging* convencional es la segunda capa de aleatoriedad a la hora de construir los distintos árboles. En un modelo *Bagging* se busca la mejor partición entre el conjunto de todas las p variables posibles, mientras que en un *Random Forest* no se tienen en cuenta todas las varibles del conjunto p, sino que se selecciona un conjunto aleatorio $m \leq p$ de variables, y este número de variables en un problema de clasificación suele ser $m = \sqrt{p}$.

Esta restricción es lo que nos permite reducir la correlación ρ entre los árboles del bosque. Matemáticamente, la varianza del promedio de B árboles de nuestro modelo, cuya varianza y correlación son σ^2 y ρ respectivamente, se define como:

$$\mathrm{Var}(\hat{f}_{rf}(x)) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2 \tag{3.4}$$

Con esto se limitan las variables disponibles para cada nodo, por lo que se reduce el valor de ρ y con ello la varianza total. Esto se conoce como decorrelación y es un proceso que un Bagging simple no puede alcanzar, siendo una de las características principales de un *Random Forest*.

Una vez se han entrenados los B árboles del modelo y llega el momento de la predicción (en nuestro caso del tráfico de red en tiempo real) de una nueva observación x, entra en juego el estimador. Este estimadorla consolida las predicciones de cada árbol $T_b(x)$ y, en el caso de la clasificación multiclase (DoS, ataques de fuerza bruta, inyecciones SQL, etc.), la predicción final será aquella que se ha repetido un mayor número de veces en las predicciones individuales de los B árboles:

$$\hat{C}_{rf}(x) = \text{mayoría}\{\hat{C}_b(x)\}_1^B \tag{3.5}$$

Este asegura que los errores aleatorios de un árbol individual (provocados por ruido o sobre-ajuste a su muestra de Bootstrap) se compensen con el resto de votos de los árboles restantes, mejorando la robustez del modelo.

3.5. La Impureza de Gini en un Random Forest

En esta sección se va a ver una métrica que mide qué tan "mezcladas"se encuentran las distintas clases en un nodo en concreto de un árbol del bosque, la impuereza de Gini. Dado un conjunto de datos con C clases, la impureza de Gini G en un nodo t de un determinado árbol se define como:

$$G(t) = 1 - \sum_{i=1}^{C}(p_i)^2 \tag{3.6}$$

Donde p_i es la probabilidad relativa de la clase i en dicho nodo. De esta manera, si G es igual a 0 se dice que dicho nodo es puro. Así mismo, la importancia de una variable se puede calcular mediante la suma de todas las disminuciones de la impureza de Gini que dicha variable produce al ser utilizada para dividir los nodos en todos los árboles del bosque, ponderada por la cantidad de muestras que pasan por dichos nodos. La fórmula para el decremento de impureza ΔI tras una división s en el nodo t es:

$$\Delta I(s,t) = G(t) - \frac{N_{t_L}}{N_t} G(t_L) - \frac{N_{t_R}}{N_t} G(t_R) \tag{3.7}$$

Donde N_{t_L} y N_{t_R} son el número de muestras en los nodos hijos, tanto a la izquierda como a la derecha, respectivamente. Entonces, la importancia final de la variable j sería finalmente la media de estas decrementaciones en todos los árboles del bosque. Esto es muy importante, ya que va a ser de gran de ayuda para identificar variables muy importantes para el modelo de clasificación, y otras en cambio, mucho más irrelevantes y que no aportan nada al modelo, incluso siendo negativas en algunos casos al introducir ruido. Esta teoría se implementará más adelante en (4.7.1) para concluir con el proceso de *feature selection*.

CAPÍTULO

4

Implementación

En esta sección vamos a aplicar lo visto en la teoría de manera práctica. Para ello se ha utilizado un set de datos del **CIC** (*Canadian Institute for Cibersecurity*), institución que nos ofrece datos de logs ya preprocesados, clasificados y con una limpieza previa para aplicarlos directamente a algoritmos de *Machine Learning* y *Deep Learning*, ahorrando bastante carga de trabajo. Los datos se encuentran disponibles en (2).

Para este trabajo es fundamental que los datos de entrenamiento sean de calidad, y, como bien se explica en el paper oficial (3), el equipo del *CIC* se ha asegurado que estos datos sean lo más abstractos posibles. Se diseñó una infraestructura de red física que a su vez se compone de dos subredes, la red atacante y la red víctima. En la red víctima se simuló un comportamiento humano completamente normal (peticiones *HTTP/HTTPS*, correos electrónicos, *FTP*, etc.) con 25 usuarios distintos a través de diferentes máquinas y sistemas operativos para que el tráfico sea lo más realista posible.

Ante dicho tráfico legítimo, se antepusieron distintos escenarios de ataque en distintos periodos de tiempo repartidos durante los 5 días. Vamos a ver más detalladamente todos los ataques realizados durante la fase de análisis (4.1), pero entre estos destacaron *DDoS*, *DoS*, fuerza bruta, infiltración, etc. Finalmente todo el tráfico (tanto benigno como maligno) fue procesado mediante la herramienta *CICFlowMeter* para obtener los datos que se han aplicado en este proyecto, pasando tráfico de red en formato *.pcap* a *.csv*, ofreciendo una base de datos muy sólida para poder aplicar algoritmos de predicción y clasificación.

4.1. Análisis Exploratorio de Datos EDA

A continuación se va a comenzar con un análisis *EDA* para comprender de una mejor manera como se organizan los datos. El objetivo de esta sección es identificar las características

esenciales y sacar conclusiones del conjunto de datos sin modificar, para posteriormente obtener una fuente mucho más limpia y menos redundante (si es que hubiese redundancia) en la fase de preprocesamiento (4.2). Sin más preámbulo, comenzamos con un breve vistazo al dataset:

	Destination Port	Flow Duration	Total Fwd Packets	Total Backward Packets	...	Idle Std	Idle Max	Idle Min	Label
0	389	113095465	48	24	...	4277541.062	16500000	6737603	BENIGN
1	389	113473706	68	40	...	4229413.12	16500000	6945512	BENIGN
2	0	119945515	150	0	...	24300000.0	60100000	5702188	BENIGN
3	443	60261928	9	7	...	0.0	0	0	BENIGN
4	53	269	2	2	...	0.0	0	0	BENIGN
...	...	...	...	...	...	...	...	...	...
2830738	443	196135	49	57	...	0.0	0	0	BENIGN
2830739	443	378424	49	59	...	0.0	0	0	BENIGN
2830740	443	161800	70	103	...	0.0	0	0	BENIGN
2830741	443	142864	50	62	...	0.0	0	0	BENIGN
2830742	443	186928	46	57	...	0.0	0	0	BENIGN

Figura 2: Estructura del Dataset CICIDS2017. Datos disponibles en (2)

Como se puede observar, los datos constan de casi 3 millones de registros, y abarcan casi 80 columnas las cuales son todas numéricas a excepción de la última, la cual es una variable categórica que etiqueta al dataset en función de su tráfico legítimo. Es importante recalcar que los datos no se encuentran disponibles en un único dataset, sino que se encuentran separados en ficheros *.csv* según el día que se tomaron, pero para este proyecto se han juntado dichos ficheros en un único cuerpo de datos.

Entre los datos se puede diferenciar información de interés para la detección y clasificación de anomalías como información básica de conexión (*Destination Port*, *Flow Duration*), cantidad de paquetes y su tamaño (*Total Fwd Packets*, *Total Length of Fwd Packets*), velocidad (*Flow Bytes*, *Flow Packets*), Flags TCP, etc. Y la etiqueta final, que denota el flujo como benigno o maligno. Entre estas columnas se resumen la mayoría de patrones clave para detectar anomalías y, por ende, ciberataques.

Otro aspecto fundamental a revisar es la gestión de valores nulos. Aquí se vuelve a resaltar el trabajo del equipo del *CIC* por su limpieza previa, ya que no se presentan valores perdidos en todo el flujo de datos. Asímismo, es una buena práctica separar los datos clasificados como benignos de los malignos, y esto se ve reflejado de la siguiente manera:

	Tipo	Número de Logs	Porcentaje
0	Benignos	2273097	80.3
1	Malignos	557646	19.7

Figura 3: Distribución de datos benignos y maliciosos

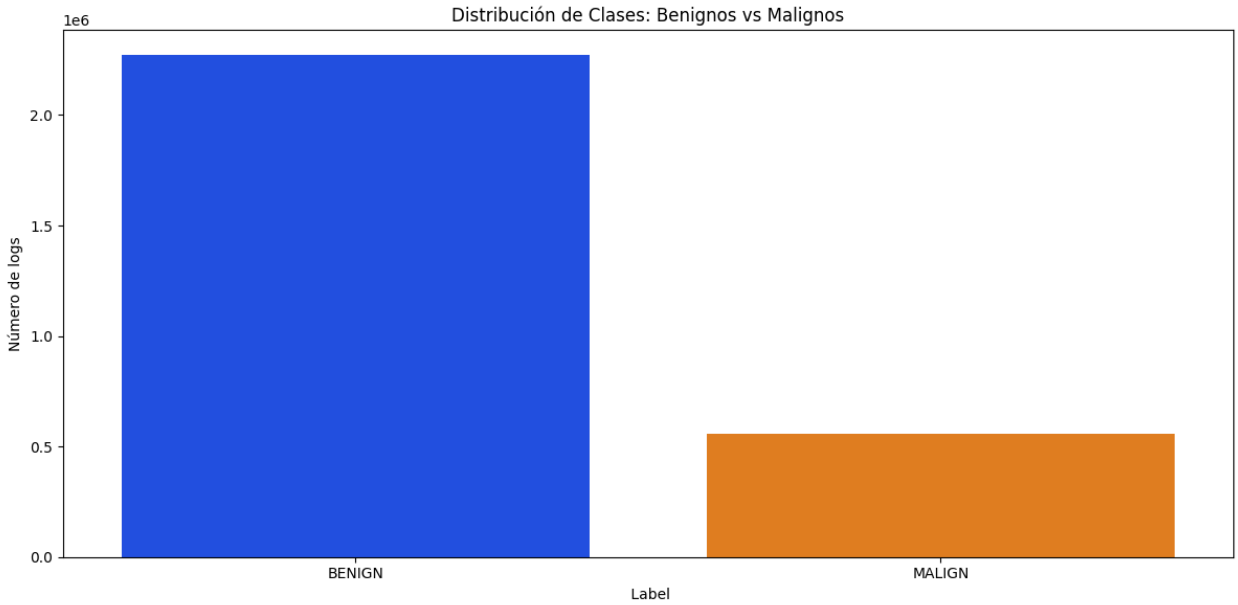

Figura 4: Distribución de clases: Datos benignos y maliciosos

Vemos claramente que aquellos datos con un fin legítimo forman un 80 % de los datos, mientras que los maliciosos un 20 %. Esto es bastante importante ya que se necesita que el autoencoder (4.3) aprenda del tráfico más usual de red para detectar anomalías con una precisión aceptable, diferenciando distintos patrones en los que se difieren los ataques. No obstante, también disponemos cerca de medio de millón de datos sobre trafico malicioso, lo que nos va a ilustrar que el autoencoder funciona correctamente detectando dicho tráfico como sospechoso, y posteriormente para entrenar un modelo de clasificación para un diagnóstico más preciso de las intenciones del atacante (4.7).

De igual manera, se ha obtenido la distribución de clase de aquellos logs maliciosos en el dataset. Esto será de gran utilidad más adelante a la hora de evaluar el modelo de clasificación (4.7) para conocer el rango de ciberataques del que se dispone y puede clasificar dicho modelo.

	Label	Número de Logs	Porcentaje
0	DoS Hulk	231073	41.44
1	PortScan	158930	28.50
2	DDoS	128027	22.96
3	DoS GoldenEye	10293	1.85
4	FTP-Patator	7938	1.42
5	SSH-Patator	5897	1.06
6	DoS slowloris	5796	1.04
7	DoS Slowhttptest	5499	0.99
8	Bot	1966	0.35
9	Web Attack Brute Force	1507	0.27
10	Web Attack XSS	652	0.12
11	Infiltration	36	0.01
12	Web Attack Sql Injection	21	0.00
13	Heartbleed	11	0.00

Figura 5: Distribución de datos maliciosos

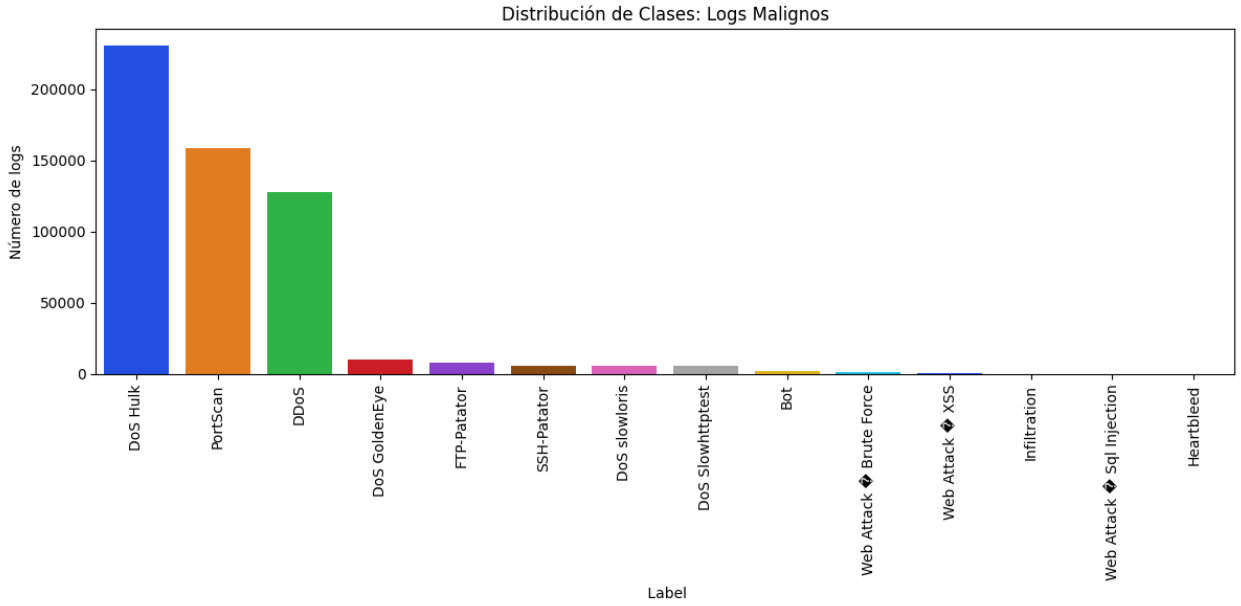

Figura 6: Distribución de clases: Datos maliciosos

Se puede apreciar que la mayoría de ataques son denegaciones de servicio, tanto denegaciones simples (DoS) como distribuidas por medio de varios sistemas o incluso una botnet ($DDoS$),

destacando también el escaneo de puertos (*PortScan*). Este patrón indica que la mayor parte del tráfico malicioso busca saturar los recursos del sistema, mientras que otros ataques menos frecuentes tienen como objetivo una intrusión remota, filtrado de datos, obtención de credenciales, etc. Comprender esta distribución de ciberataques es un aspecto fundamental para diseñar la red y posteriormente un modelo de clasificación que funcione adecuadamente detectando la amenaza que corresponda según el contexto del servidor.

Por otra parte, no es posible llevar a cabo un *EDA* completo sin realizar un análisis de *outliers*. Un *outlier* o también conocido como valor atípico, es un dato que se aleja muy significativamente del patrón del resto de datos, por lo que afectan negativamente al rendimiento de los modelos, no solo nuestro futuro autoencoder, sino cualquier modelo de *Machine Learning* se ve afectado por esto, bien un modelo de regresión, clasificación, series temporales, etc. Es por esto que los *outliers* en ámbitos relacionados con la inteligencia artificial se suelen suprimir, aunque, como se va a ver a continuación, esto no siempre es así, y se ha de analizar de la misma manera el contexto de los datos.

Para hallar estos valores, hemos de encontrar el umbral inferior y superior de los datos. Todos los datos contenidos entre estos umbrales son datos convencionales, perfectos para entrenar modelos. No obtante, si se encuentran fuera de estos límites, son considerados atípicos y se han de gestionar más específicamente:

$$U_i = Q1 - 1{,}5\, IQR \tag{4.1}$$

$$U_s = Q3 + 1{,}5\, IQR \tag{4.2}$$

Donde $Q1$ y $Q3$ son el primer y tercer cuartil respectivamente, donde IQR es el rango intercuartílico, siendo U_i y U_s el límite inferior y superior. Con esto, se han obtenido los siguientes *outliers*:

	Columna	Número de Outliers	Porcentaje
0	Destination Port	627245	22.158317
1	Flow Duration	532586	18.814354
2	Total Fwd Packets	332693	11.752851
3	Total Backward Packets	308291	10.890816
4	Total Length of Fwd Packets	446972	15.789918
...	...	...	...
47	Subflow Bwd Bytes	619046	21.868675
48	Init_Win_bytes_forward	439029	15.50932
49	Init_Win_bytes_backward	285710	10.09311
50	act_data_pkt_fwd	310279	10.961045
51	min_seg_size_forward	2149	0.075916

Figura 7: Dataframe de outliers

Vemos que la presencia de valores atípicos es muy grande. Sin embargo no hay que preocuparse, pues esto es normal. El tráfico de una red es muy estocástico, y los datos generalmente son muy asímetricos (los datos no se ajustan siguiendo una campana de Gauss). No obstante obtenemos demasiados outliers, en parte por lo que se acaba de explicar, y porque los datos maliciosos provenientes de un ciberataque son ya por definición outliers por sí solos, aumentando aún más el número final. A continuación se presentan ciertos gráficos que apoyan estos puntos:

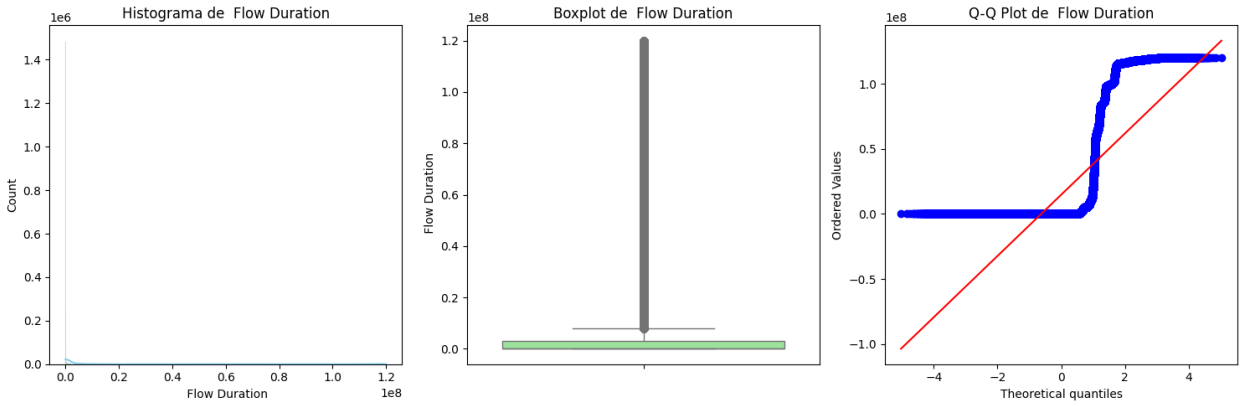

Figura 8: Histograma, boxplot y Q-Q plot sobre la variable 'Flow Duration'

En esta prueba vemos tres gráficas, un histograma, un diagrama de caja y un Q-Q plot, dichos gráficos realizados sobre la variable 'Flow Duration' del dataset, ya que esta columna es muy representativa para el modelo, al ser la duración un aspecto fundamental para explicar

el comportamiento de los datos. En el histograma se puede ver que los datos no siguen una campana de Gauss, ya que los datos se encuentran extremadamente sesgados a la derecha, donde la inmensa mayoría de registros tienen un valor muy cercano a cero, y la gran minoría restante forman una cola hasta $1{,}2 \times 10^8$.

Además, el diagrama de caja refuerza los datos que muestra el histograma, ya que la propia caja del diagrama se encuentra muy cercana a 0 (donde residen la mayoría de datos) y se muestra una gran cantidad de atípicos por encima del límite superior. Por último, el diagrama Q-Q plot termina de reforzar la teoría, viéndose claramente que los datos no siguen una distribución normal, sino que se agrupan reflejando gran cantidad de outliers.

Con esto queda justificado la presencia de multitudes valores atípicos en los datos, ya que los datos del tráfico de una red se encuentran demasiado sesgados y dispersos, negándose a seguir una distribución normal. Es por esto que se implementa un modelo autoencoder, al presentar una arquitectura no lineal que se ajusta mucho mejor a los datos, pudiendo aprender a partir de su comportamiento y características sin depender de supuestos gaussianos.

Una vez se ha visto el análisis de valores atípicos, hemos de comprobar que las distintas columnas no estén muy relacionadas entre sí, ya que el futuro modelo sería redundante y estaríamos procesando información de más que no aporta nada a la red. Para ello, obtenemos la matriz de correlación del dataset:

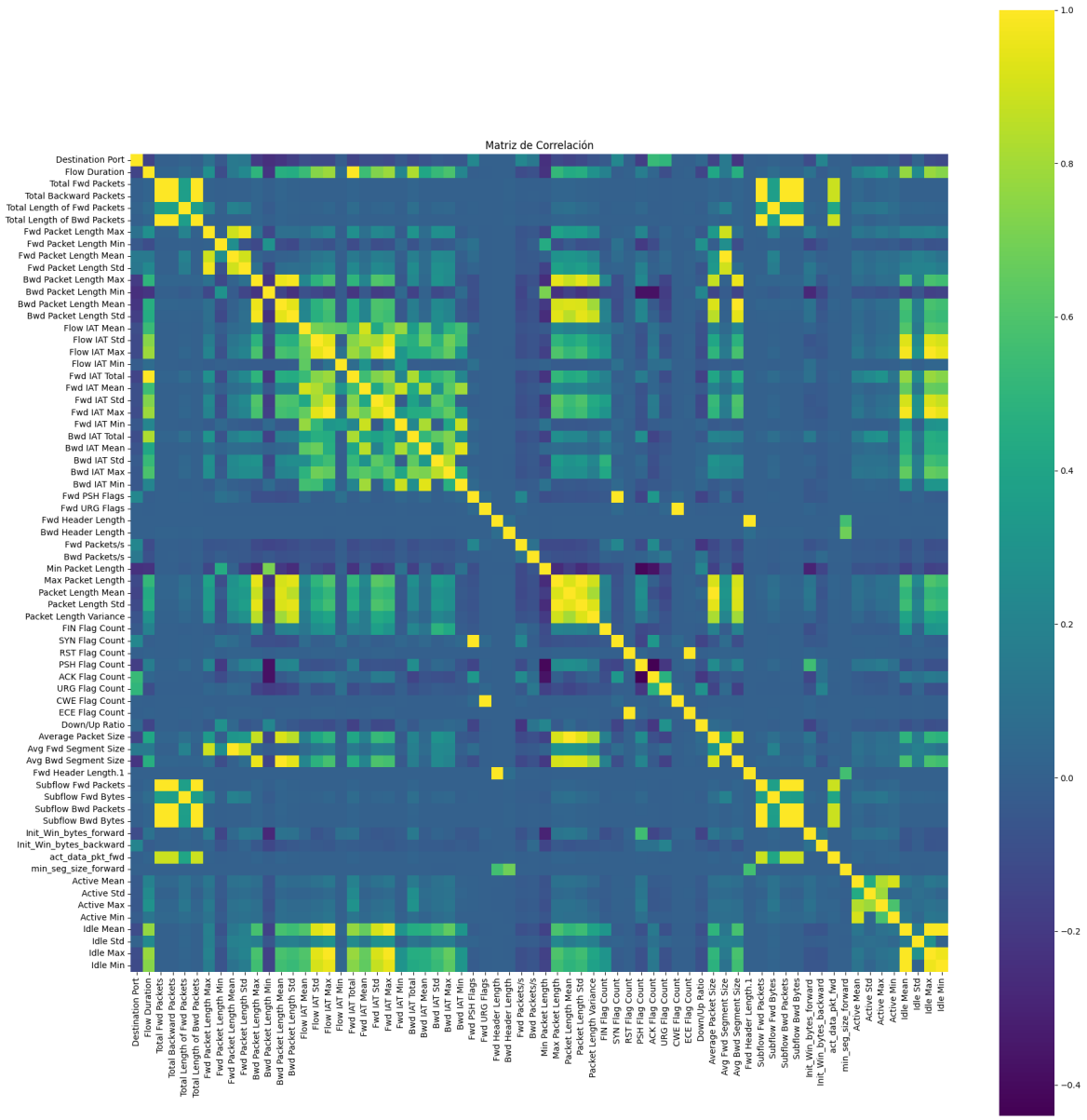

Figura 9: Matriz de correlación inicial de los datos

La matriz de correlación es muy densa, pues el dataset contiene muchas columnas de datos. Sin embargo esto no nos impide verificar la presencia de variables altamente correlacionadas entre sí. Estas distintas columnas, especialmente aquellas con una correlación muy cercana a la unidad, generan mucha redundancia al modelo, algo que no es conveniente, por lo que será uno de los primeros pasos que seguiremos en la fase de procesamiento (4.2). Con esto se da por finalizado el análisis exploratorio de datos EDA y se da paso a la sección que se acaba de mencionar.

4.2. Preprocesamiento

Una vez se ha comprendido de una mejor manera la naturaleza de los datos y como se distribuyen mediante un análisis exploratorio (4.1), podemos dar paso a la fase de procesamiento. El objetivo de esta sección simplemente es transformar el conjunto de datos de la manera más óptima posible para entregarlos a la capa de entrada de nuestro autoencoder. En esta fase se llevará a cabo un filtrado de variables altamente correlacionadas en los datos para reducir la redundancia de la red, y posteriormente se normalizarán los datos para mejorar el rendimiento del modelo y que se aprendan las características del tráfico benigno de una red correctamente.

Antes de comenzar, es importante saber que se han eliminado ciertas variables del dataset. El problema de estas variables es que no es posible obtener datos en tiempo real de ellas a partir de la librería utilizada, *NFStream*. Esta librería permite obtener gran cantidad de información del flujo de una red, aunque se ha prescindido de ciertos apartados que esta librería no proporciona. Entre estas variables destacan aquellas relacionadas con el conteo de paquetes por ráfagas (*Bulk*) y con el tamaño de ventanas TCP (*Window*).

Como se ha explicado, vamos a comenzar con la gestión de variables altamente relacionadas. Esto está muy ligado con el final de la sección anterior (4.1) en el que obtuvo la matriz de correlación inicial de los datos (9) pero no se llegó suprimir las variables muy correlacionadas. Para ello, se seleccionó el triángulo superior de la matriz, eliminando variables que se generaban en función de otras. Un ejemplo de esto son las columnas *'Total Length of Fwd Packets'* y *Subflow Fwd Bytes*, donde la segunda variable se obtiene a partir de la longitud total de los paquetes enviados. Así, eliminamos una de las dos variables altamente correlacionadas para reducir la redundancia, para aquella correlación superior o igual a 0,95.

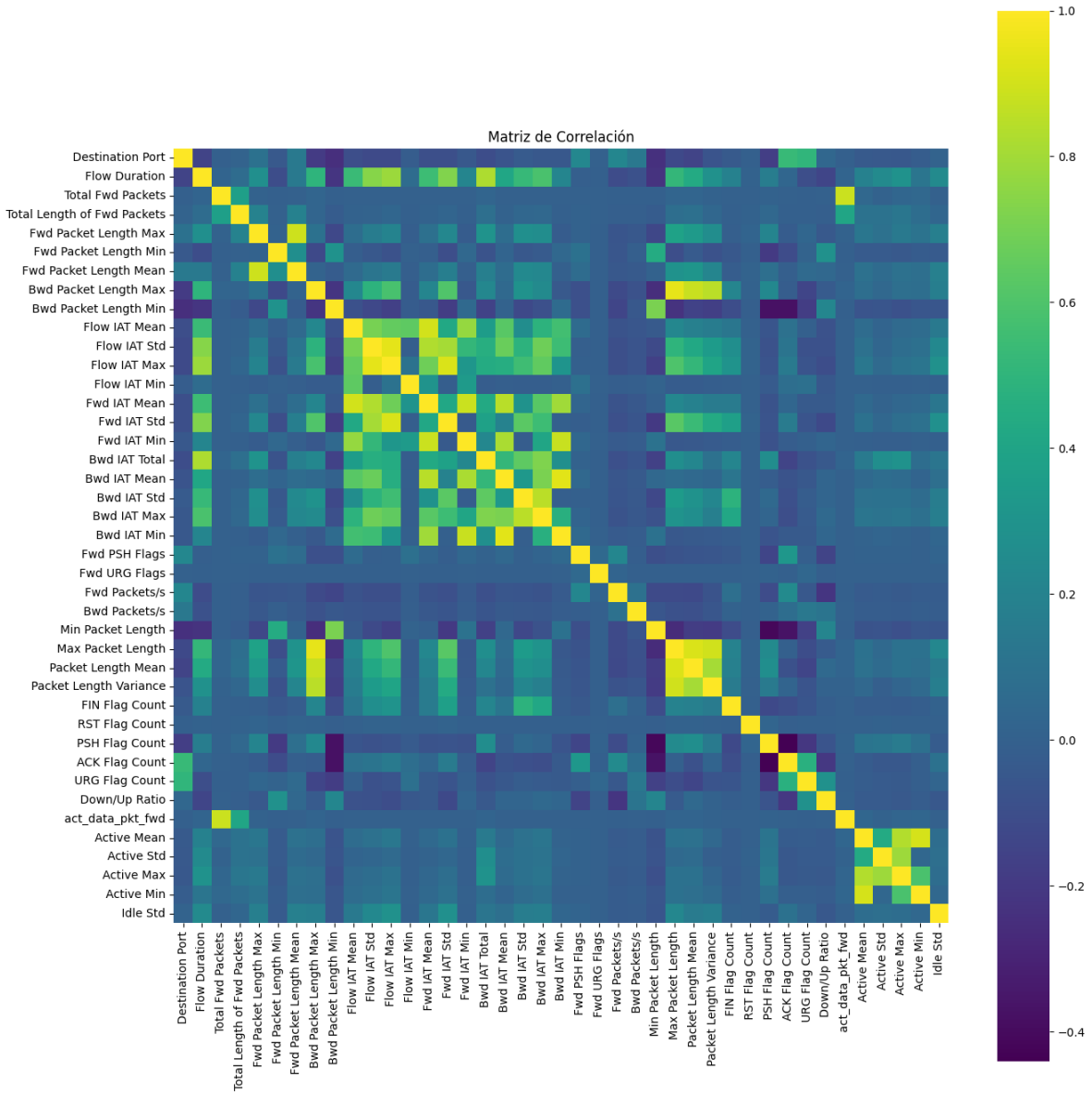

Figura 10: Matriz de correlación preprocesada de los datos

Se puede apreciar que la matriz de correlación presenta una limpieza mucho más considerable, pues se han eliminado 22 columnas repetitivas de los datos (30 en total si contamos las eliminadas antes de obtener la matriz de correlación), mejorando el rendimiento de la red. No obstante, para evitar sesgos de red y un mejor funcionamiento del modelo de clasificación de ataques maliciosos (4.7), se ha reducido aún más las características de los datos, eliminando las siguientes variables y grupos de variables:

Variable(s)	**Justificación**
Flow Duration	La duración en entrenamiento corresponde a un flujo completo, mientras que al capturar en tiempo real se tiene un flujo parcial, ya que se desconoce la finalización del ataque. Esta discrepancia introduce errores al modelo.
Flow Bytes/s y Flow Packets/s	Métricas dependientes del rendimiento del hardware (CPU/NIC) y de la carga de la red en un momento dado. Su variabilidad impide una firma constante del ataque.
Flow IAT (Mean, Max, Std, Min)	El tiempo de llegada entre paquetes (Inter-Arrival Time) se ve alterado por la latencia de procesamiento del sistema de captura en vivo, introduciendo ruido que no existe en el dataset estático.
Active / Idle (Mean, Max, Min, Std)	Parámetros puramente temporales que describen el comportamiento de la sesión pero no la naturaleza maliciosa del paquete. Generan un fuerte sesgo hacia el entorno de laboratorio.

Cuadro 1: Variables eliminadas del modelo para mejorar la generalización en tiempo real.

Con esto se evita que se memorice la velocidad de los paquetes en los datos de entrenamiento, con el fin de identificar el tipo de ataque malicioso únicamente en función de su comportamiento lógico, lo que incluye flags TCP, tamaño y métricas estadísticas de paquetes, etc. Adicionalmente con la implementación anterior, evitamos toda dependencia al hardware de la red, al no registrarse parámetros de velocidad de paquetes, causando un funcionamiento mucho más homogéneo de este proyecto al ser completamente independiente de la red donde se analice el tráfico en tiempo real. Finalmente, la matriz de correlación final se muestra a continuación:

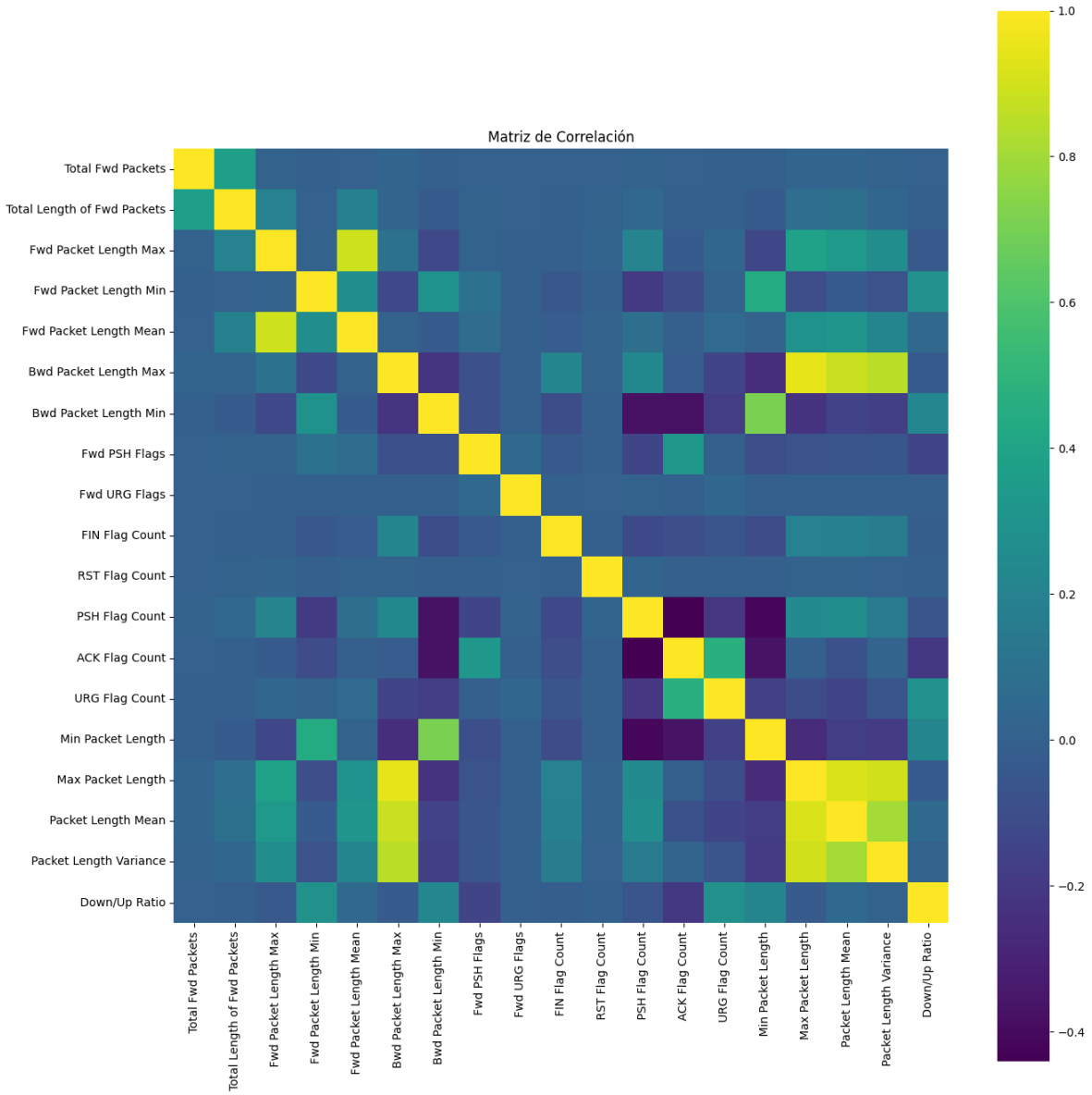

Figura 11: Matriz de correlación final

Por otra parte, cabe mencionar que se encontraron valores infinitos en el dataset, los cuales por supuesto han sido correspondiente eliminados, ya que un valor infinito no es válido para una red neuronal, ahorrando futuros problemas con nuestro autoencoder.

La segunda parte de esta breve sección de preprocesamiento consiste en estandarizar los datos. Inicialmente los datos fueron normalizados entre 0 y 1, pero debido a diversos problemas que se tuvo a la hora de identificar anomalías, se ha optado por un *Standard Score*. Este enfoque

transforma las variables para que presenten una media de 0 y una desviación estándar de 1, lo que permite que el modelo detecte mucho mejor los valores atípicos (vista su alta presencia anteriormente en (4.1)) al encontrarse ráfagas muy intensas de datos (propias de un ciberataque) que se alejan muy significativamente de la media y aumentando muchísimo el error en la reconstrucción.

```
def get_scaled_train_test_data(df_to_scale):
    scaler = StandardScaler()
    X_train, X_test = train_test_split(
        df_to_scale,
        test_size=0.20,
        random_state=42
    )
    # Normalizamos los datos para que tengan valores entre 0 y 1
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)
    return X_train_scaled, X_test_scaled
```

Con esta función nos estamos ayudando de la librería Scikit-Learn de Python, disponible en (4). Internamente, se está aplicando la siguiente transformación para el escalado de cada uno de los datos:

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma} \tag{4.3}$$

Donde μ representa la media aritmética de las muestras de entrenamiento y σ la desviación típica. Este proceso se repite para cada columna del dataset, dejando completamente escalados los datos. Con el código anterior también dividimos los datos para el entrenamiento y test, dejando perfectamente preparados los datos para entrenar la red y verificar su correcto funcionamiento, dando paso a la siguiente sección.

4.3. Autoencoder

Según se vio en (3.3), el autoencoder consta de 3 capas: La capa de entrada de datos que aplica una función f, obteniendo las características principales para replicar los datos en una capa intermedia, para posteriormente aplicar la función g y replicar los datos de entrada en la capa de reconstrucción o salida de datos. Esto se ve reflejado en el siguiente código:

```
def create_and_train_autoencoder(X_train_scaled, X_test_scaled):
    # Creamos la red neuronal (autoencoder)
```

```
input_dim = X_train_scaled.shape[1]
entrada = Input(shape=(input_dim,))
encoder = Dense(int(input_dim/2), activation='relu')(entrada)
decoder = Dense(input_dim, activation='linear')(encoder)

autoencoder = Model(inputs=entrada, outputs=decoder)
autoencoder.compile(optimizer='adam', loss='mse')

print('Comenzamos el entrenamiento de la red...')
history = autoencoder.fit(
    X_train_scaled,
    X_train_scaled,
    epochs=20,
    batch_size=256,
    validation_data=(X_test_scaled, X_test_scaled),
    callbacks=[early_stop],
    verbose=1
)
return autoencoder, history
```

Donde vemos que las funciones de activación f y g son las funciones *ReLu* y la función lineal respectivamente. En primer lugar se usa la función **ReLu** ya que es la función por excelencia en capas ocultas al solucionar el desvanecimiento de gradiente, y al ser lineal en los valores positivos permite a la red aprender muy rápido teniendo en cuenta las características más relevantes del tráfico de red, debido a que los valores negativos son cero.

$$f(x) = \max(0, x) \tag{4.4}$$

Por otra parte, se ha optado por una función de activación lineal para la capa de salida. Normalmente se usaría la función Sigmoidea ya que la salida de las redes suele estar normalizada. Sin embargo, debido a la alta presencia de valores atípicos, este no es nuestro caso. Dado que nuestros datos han sido estandarizados mediante un *StandardScaler*, los valores de la capa de entrada, y por lo tanto también los de reconstrucción, ya no se encuentran sujetos a escala $[0, 1]$, sino que pueden alcanzar valores significativamente superiores al toparse con cualquier anomalía. De esta manera una función lineal permite que las neuronas de salida repliquen cualquier valor sin sufrir el efecto de saturación que provocarían funciones como la sigmoide, lo que resulta fundamental para que el error cuadrático medio (MSE) refleje con precisión la distancia estadística entre el tráfico benigno y un ataque malicioso masivo.

$$g(x) = x \tag{4.5}$$

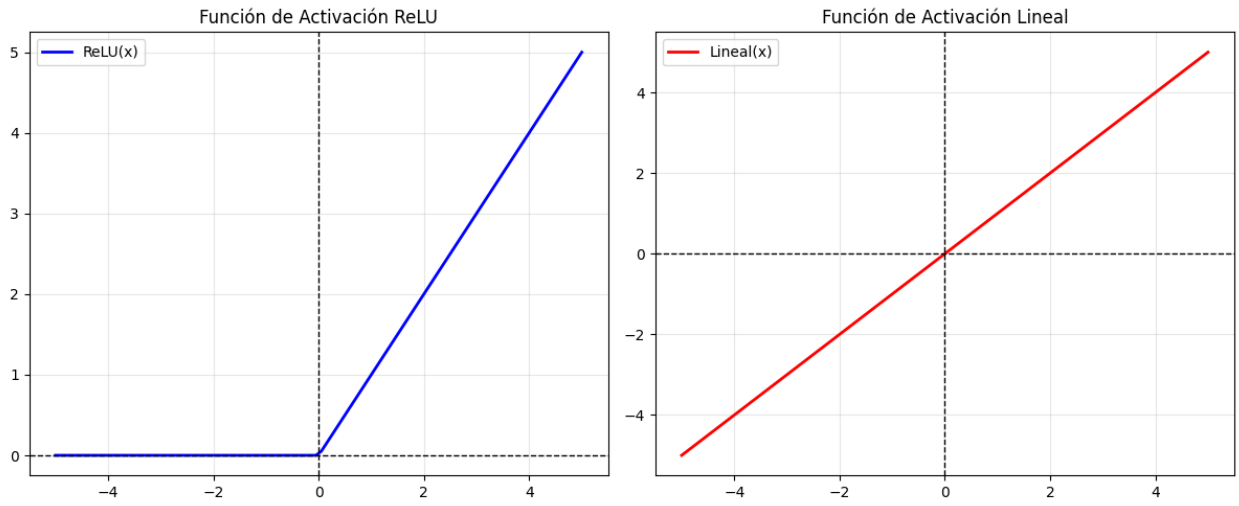

Figura 12: Función ReLu y Lineal

Una vez la fase de entrenamiento de la red ha finalizado se puede finalizar con la conclusión de los resultados. Como se vio anteriormente en (3.2), nos interesa conocer el error **MSE**, ya que en función de dicho valor se va a detectar un ataque con intenciones maliciosas. Para ello, se ha reservado un conjunto de datos clasificados como benignos (los cuales no han participado en la fase de entrenamiento de la red) y otros maliciosos, obteniendo distintas tasas de error:

	Tipo	MSE
0	Benigno	0.329209
1	Maligno	1.452152

Figura 13: Comparación de MSE para datos benignos y maliciosos.

Vemos que los resultados son lo que se esperaba, siendo el error casi 5 veces mayor para datos maliciosos. Esta es la funcionalidad principal del autoencoder, detectar el tráfico legítimo de paquetes de red a partir de los valores del error del modelo. Con esto concluye esta sección, una sección de vital importancia para validar la capacidad de selección de la red neuronal, verificando la correcta capacidad de aprendizaje del modelo y estableciendo la base necesaria que actúa como umbral para la detección de intrusiones y tráfico malévolo en tiempo real. En la sección de demostración (**??**) veremos que el error al realizar ataques como *DoS* se dispara mucho más que el valor obtenido en la tabla anterior (13) ante la presencia de ataques maliciosos muy volumétricos.

4.4. Servidor Web Apache

En esta fase se levanta un servicio web con **Apache2** sobre un sistema operativo Ubuntu. En este apartado se mantuvo la página web de apache por defecto así como su configuración inicial. El objetivo de esta sección fue validar la conexión con el servidor desde una máquina en su misma red con el fin de monitorizar el tráfico hacia nuestro servidor apache desde dicho sistema, detectando ciberataques en tiempo real en función de los paquetes de red enviados.

Durante esta etapa se probaron distintos ataques para verificar que el modelo detecta el tráfico inusual de red correctamente, detectando así tráfico malicioso hacia el servidor en tiempo real. El primer caso que se probó fue una denegación de servicio (**DoS**) a través de la herramienta **hping3**, generando así paquetes TCP/IP de gran tamaño y a muy alta velocidad con el fin de saturar el servidor. Es importante destacar que el servidor no quedó obsoleto por el número de paquetes debido a las características modernas de apache, aunque lo primordial con esta prueba es detectar el ciberataque antes de que tenga éxito para proteger nuestro servidor. Podemos encontrar documentación acerca de esta herramienta en (5), poniendo como ejemplo el siguiente comando para reproducir el envío masivo de paquetes:

```
sudo hping3 -S -p 80 -s 5050 -i u100 127.0.0.1
```

Con esto producimos una inundación (*flooding*) de paquetes SYN cuya finalidad es agotar los recursos del servidor, causando la DoS. Las pruebas demostraron que esta anomalía se detecta correctamente por nuestro autoencoder (3.2), debido al extraño flujo de red que esto genera, se alcanzaron flujos con mas de 100000 paquetes, disparando la cifra del MSE:

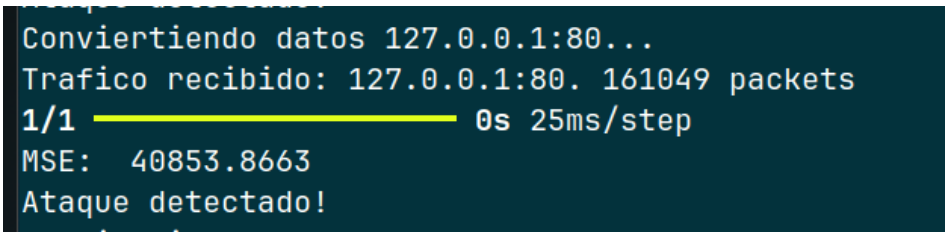

Figura 14: Captura de flujo malicioso en tiempo real causado por hping3.

Otros puntos a mencionar son las limitaciones de la página web de apache por defecto, ya que no permite reproducir muchos ciberataques. Al ser un HTML estático no podemos realizar una conexión con una base de datos para validar formularios de inicio de sesión o un filtro de búsqueda, lo cual es ideal para ataques de fuerza bruta e inyecciones SQL respectivamente. Para ello, se desarrolló una página web personalizada para incluir estos elementos, funcionando como una tienda de electrónica (sin implementar el sistema de venta).

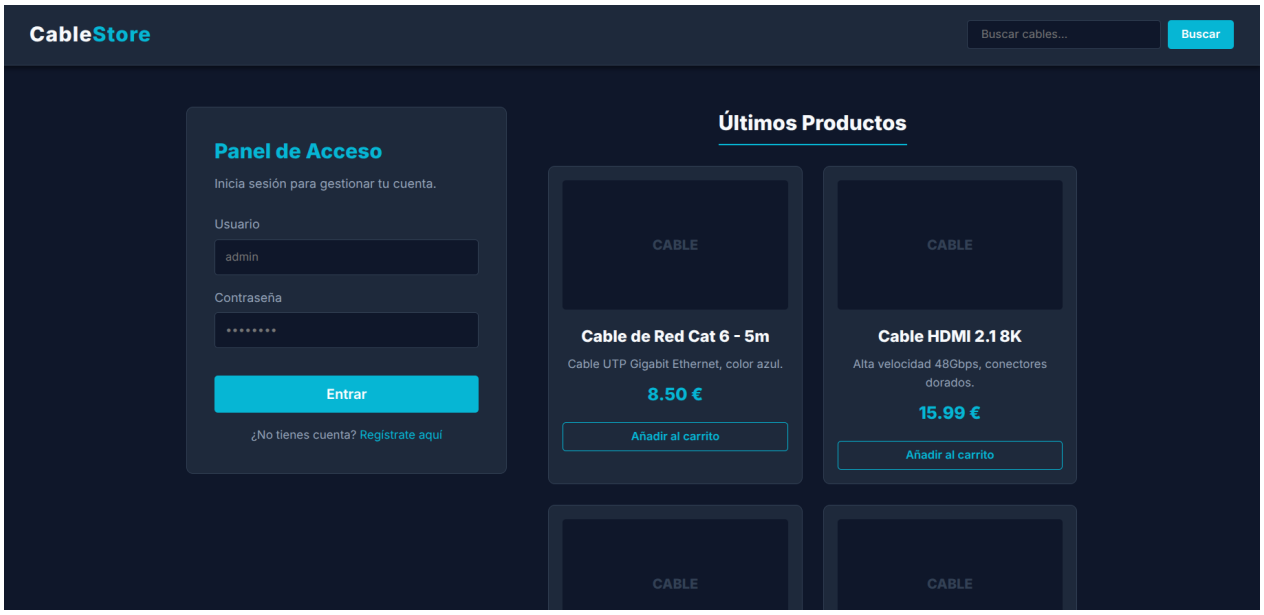

Figura 15: Pagina personalizada utilizada para el servidor web con Apache

Es por esto que se migró a una arquitectura **LAMP** (Linux, Apache, MariaDb, PHP), permitiendo la conexión de una base de datos SQL como **MariaDb** a partir de un servidor Apache en un sistema Linux a través de PHP, implementando el formulario de inicio de sesión y el buscador de inventario, mediante los cuales realizaremos distintos ataques para verificar el correcto funcionamiento del autoencoder. En cuanto a la base de datos MariaDb, se crearon dos tablas para gestionar los datos de la web, manejando de una mejor manera los datos de inicio de sesión con credenciales y los productos en el inventario, como se puede ver a continuación:

```sql
CREATE DATABASE tfg_2026;
USE tfg_2026;

CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL,
    password VARCHAR(255) NOT NULL
);

CREATE TABLE products (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    price DECIMAL(10, 2)
);
```

De esta manera se crea la base de datos y sus correspondientes tablas para almacenar la información. Posteriormente, se procede a insertar registros de prueba para simular un inventario de cables, con el fin de validar las funciones que se implementan en la web y sus contrapartidas maliciosas:

```
INSERT INTO users (username, password)
VALUES ('admin', 'admin_tfg_uax');

INSERT INTO products (name, description, price) VALUES
('Cable de Red Cat 6 - 5m', 'Cable UTP Gigabit Ethernet, color
↪  azul.', 8.50),
('Cable HDMI 2.1 8K', 'Alta velocidad 48Gbps, conectores dorados.',
↪  15.99),
('Cable USB-C a USB-C 100W', 'Cable trenzado para carga rápida y
↪  datos.', 12.00),
('Riser PCI-Express 4.0', 'Cable extensor flexible para GPU.',
↪  29.90),
('Cable DisplayPort 1.4', 'Soporta 144Hz a 4K, ideal gaming.',
↪  18.50),
('Fibra Óptica LC a LC - 10m', 'Cable de parcheo multimodo dúplex.',
↪  24.00);
```

Y por último, por motivos de seguridad, se creó un usuario en específico para realizar consultas a través de la página PHP, con permisos limitados para evitar el uso de permisos de administrador sobre el sistema:

```
CREATE USER 'admin_tienda'@'localhost' IDENTIFIED BY 'admin_tfg_uax';

GRANT ALL PRIVILEGES ON tfg_2026.* TO 'admin_tienda'@'localhost';
FLUSH PRIVILEGES;
```

Con esto ya se tiene una infraestructura vulnerable para continuar realizando pruebas hacia nuestro servidor web, capturando el tráfico de red en tiempo real mediante NFStream para evaluar el rendimiento del autoencoder (4.3). La segunda prueba que se va a realizar son las inyecciones SQL con la herramienta **SQLMap**.

4.5. Pruebas de Penetración con SQLMap e Hydra

SQLMap es una herramienta de código abiero que automatiza las inyecciones SQL, buscando vulnerabilidades que puedan ser explotadas con un tipo de inyección en específico. Aplicando la herramienta a este proyecto, se realizó un escaneo muy exhaustivo sobre el buscador de

inventario de la web, con el fin de hacer un volcado de la base de datos, a través de la siguiente instrucción:

```
sqlmap -u "http://192.168.1.50/index.php?search=cable"
↪ --random-agent --level=5 --risk=3 --dump  --flush-session
```

Con esto generamos una gran cantidad de payloads sobre el buscador de elementos del servidor, lo que se traduce en un flujo de red con características muy extrañas (número de paquetes muy elevado con un tamaño reducido, alta freucencia de peticiones, etc.) que son detectadas por el autoencoder pefectamente, alcanzando un MSE muy elevado como se justifica en la siguiente imagen:

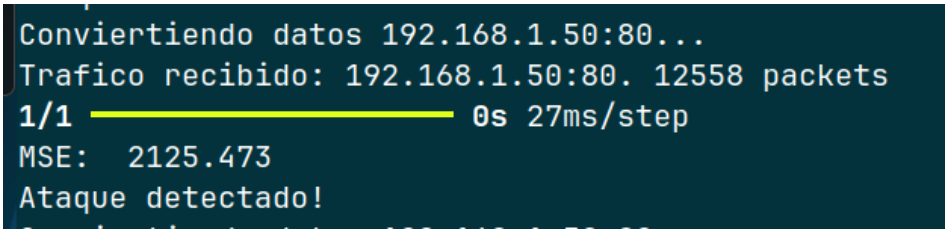

Figura 16: Captura de flujo malicioso en tiempo real causado por SQLMap

Se puede apreciar perfectamente que el resultado es el esperado, viéndose multiplicado el error con respecto a un tráfico legítimo por varios centenares. Con esta prueba verificamos que también se detecta el tráfico malicioso generado por herramientas que automatizan inyecciones SQL, como es el caso de SQLMap. No obstante, se va a realizar una prueba adicional para verificar el correcto funcionamiento del modelo, en este caso utilizando la herramienta profesional especializada en ataques de fuerza de bruta por excelencia, **Hydra**.

Hydra es una de las herramientas de crackeo de credenciales más rápidas y versátiles que existen. El objetivo enfocado a este proyecto es obtener las credenciales del usuario admin creado con las consultas SQL de la sección anterior (4.4) a través de un ataque de diccionario, el cual se ha empleado el famoso rockyou.txt, uno de los diccionarios más famosos en el ámbito de la ciberseguridad, ya que contiene gran cantidad de credenciales filtradas a través de internet.

```
hydra -l admin -P ~/rockyou.txt 192.168.1.50 http-post-form
↪ "/index.php:username=^USER^&password=^PASS^:Inicio de sesión
↪ fallido" -t 4
```

Este comando nos genera un pico de peticiones POST sobre el formulario de inicio de sesión en un intervalo muy corto, capturando el siguiente error por la herramienta NFStream a partir del flujo de red en tiempo real:

```
Conviertiendo datos 192.168.1.50:80...
Trafico recibido: 192.168.1.50:80. 2448 packets
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step
MSE:  36.182
Ataque detectado!
```

Figura 17: Captura de flujo malicioso en tiempo real causado por Hydra

Sin embargo, pese a que el error es notablemente más alto que los datos de entrenamiento, tampoco alcanza valores extremos como en el resto de pruebas. Esto es porque Hydra genera un patrón de tráfico estructurado y repetitivo, lo que reduce el número de solicitudes enviadas al servidor, desencadenando en un MSE notablemente mayor pero sin llegar a valores tan altos como en las pruebas anteriores. Por lo tanto, se ha buscado una alternativa mediante un script personalizado de Python para lograr un control total sobre las peticiones de un ataque de fuerza bruta:

```python
import requests
import os
from dotenv import load_dotenv

load_dotenv()
url = "http://192.168.1.50/index.php"
username = "admin"
password_file = os.getenv('ROCKYOU_PATH')

print(f"[*] Iniciando ataque de fuerza bruta sobre {url}...")

with open(password_file, "r") as f:
    for line in f:
        password = line.strip()
        data = {
            "username": username,
            "password": password
        }

        if "Inicio de sesión fallido" not in response.text:
            print(f"[+] Éxito: {password}")
            break
```

```
Conviertiendo datos 192.168.1.50:80...
Trafico recibido: 192.168.1.50:80. 27126 packets
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step
MSE:  10614.7312
Ataque detectado!
```

Figura 18: Captura de flujo malicioso en tiempo real a través de Python

Y de esta manera se presencia un error de reconstrucción mucho mayor que el anterior, en parte debido a la mayor presencia de paquetes en el flujo, ya que se envían los paquetes en crudo y sin ningún procesamiento a través de la librería **requests** de Python, a diferencia de Hydra, la cual optimiza los paquetes para ser más eficientes y reducir cabeceras HTTP con el fin de reducir el rastro del ataque. Con estos hallazgos, se dan por cerradas las pruebas de penetración en las que se han validado tanto herramientas automatizadas open-source como profesionales, además de scripts atacantes codificados a medida para penetrar el servidor web.

4.6. Determinación del Umbral de Detección de Anomalias

Una vez ya se han realizado las pruebas de penetración anteriores y el modelo responde correctamente, se ha de determinar el umbral. Este umbral (τ) es la frontera que separa el tráfico benigno del malicioso, lo que en la práctica se traduce como el valor mínimo de MSE para detectar tráfico con intenciones maliciosas. Lo correcto es que este valor no sea escogido a la ligera, sino que se analicen distintos métodos para su obtención, comprobando el mecanismo más robusto para su implementación en nuestro sistema de detección de intrusiones en tiempo real.

En primera instancia, se tomaron valores reales de tráfico de paquetes en una red doméstica. Se pudo haber usado directamente los datos de entrenamiento, pero consideramos que esto es una mejor opción debido a la variabilidad de un entorno real, que frente a la naturaleza estadística de nuestros datos de entrenamiento, incluyen ruido del sistema operativo, latencia de red, diferencias en las cabeceras HTTP, etc. Es por esto que se almacenaron datos del MSE como resultado de pasar dichos flujos de red por el autoencoder, flujos realizados desde distintos dispositivos y cuyo archivo se encuentra en el repositorio de Github de este trabajo (6).

Por ello, se implementaron tres distintas metodologías con el fin de que dicho umbral sea lo más robusto posible, minimizando falsos positivos pero sin llegar a comprometer el sistema ante ciberataques más sigilosos (como la prueba realizada con Hydra anteriormente en (17)).

4.6.1. Método de la Desviación Estándar

En este método asumimos que el MSE sigue una distribución normal. Entonces, podemos definir el umbral en función de la media (μ) y la desviación estándar (σ):

$$\tau = \mu + (k \cdot \sigma) \tag{4.6}$$

Donde k representa un factor de sensibilidad. En función de este valor se puede ser más conservador y abarcar prácticamente todo el tráfico benigno en su totalidad, o se puede tomar un enfoque más radical. No obstante, este último enfoque no es el más adecuado ya que generaría muchos falsos positivos. Inicialmente se propuso un valor $k = 3$ para las primeras pruebas, aunque más adelante se modificó este valor hasta $k = 5$ para garantizar un entorno altamente conservador. Con este dato y conociendo el valor de μ y σ se puede hallar el umbral en función de la desviación estándar como sigue:

$$\tau = \mu + (k \cdot \sigma) = 1{,}4366 + (5 \cdot 1{,}5654) = 9{,}2634$$

Con este método logramos obtener un umbral de $\tau = 9{,}2634$, el cual veremos que es el más conservador de los tres modelos de umbrales que se va a ver.

4.6.2. Método de la Percentiles

Este segundo enfoque, a diferencia del modelo anterior (4.6.1), no asume una distribución en específico, sino que se centra en la estadística, en el percentil P, lo que indica que el $P\,\%$ de los datos se encuentran por debajo del umbral, como sigue:

$$\tau = P_{99{,}9}\,(\mathrm{MSE}_{benigno}) = 6{,}5387 \tag{4.7}$$

Sin embargo, como se puede observar, este valor es inferior al anterior, y debido al enfoque conservador que se quiere emplear en este apartado, se ha decidido finalmente por descartar este método, para disminuir al máximo la posibilidad de obtener falsos positivos al analizar el flujo de red.

4.6.3. Método del Valor Máximo

Esta última metodología consiste en identificar el valor máximo de los datos de prueba capturados previamente, añadiendo un margen de guarda para establecer el umbral:

$$\tau = \max\left(MSE_{benigno}\right) + \Delta \tag{4.8}$$

Sin embargo este método tampoco terminó de encajar, ya que depende mucho del margen de guarda que se escoja. Es por esto que se decidió implementar el método de la desviación estándar (4.6.1) con un valor de $k = 5$, ya que a diferencia de este último método, se tiene en cuenta la distribución de los datos, lo que permite ignorar picos de ruido de forma puntual mientras se mantiene la capacidad de detección de ciberataques como las pruebas de penetración vistas anteriormente en (4.5), manteniendo un enfoque conservador y disminuyendo los falsos positivos.

$$\boxed{\tau = 9{,}2634}$$

4.7. Modelo de Clasificación

A continuación se va a aplicar la teoría propuesta en (3.4) a la práctica, mediante la implementación de un *Random Forest*. Para ello se ha utilizado la librería `scikit-learn` (4), seleccionando el algoritmo `RandomForestClassifier` con los siguientes hiperparámetros para equilibrar las capacidades de detección del modelo y previniendo el sobreajuste (*overfitting*):

```
rf = RandomForestClassifier(
        n_estimators=300,
        max_depth=20,
        min_samples_split=5,
        random_state=42,
        n_jobs=-1
    )
```

Donde `n_estimators=300` define el número de árboles de decisión en el bosque. Un valor entre 100 y 300 permite reducción significativa de la varianza que se vio anteriormente en (3.4.1), lo que estabiliza el error de predicción sin muchas limitaciones de cómputo a nivel de hardware. Por otra parte `max_depth=20` limita la profundidad máxima de los árboles, lo que ayuda mucho a su regularización, evitando ruido y forzando a aprender patrones más

generales. `min_samples_split=5` establece el número mínimo de muestras necesarias para dividir un nodo internamente en cualquiera de los árboles del bosque, mejorando la robustez, `random_state=42` fija la semilla de aleatoriedad, con lo que nos aseguramos que el proceso de *bootstrap* y la selección de características sean exactamente iguales en cada ejecución, y `n_jobs=-1` simplemente habilita el procesamiento en paralelo para acelerar el entrenamiento del modelo.

4.7.1. Importancia de Gini para la selección de Variables

En esta sección se va a justificar la eliminación de ciertas variables de los datos, ya que anteriormente en (1) se eliminaron como medida previa al entrenamiento de la red neuronal, explicando a medias el razonamiento. Es aquí, gracias a la importancia de Gini que se puede justificar este cambio por completo. Inicialmente, nuestros datos parten de la siguiente manera (habiendo hecho un análisis de correlación previamente en (10)):

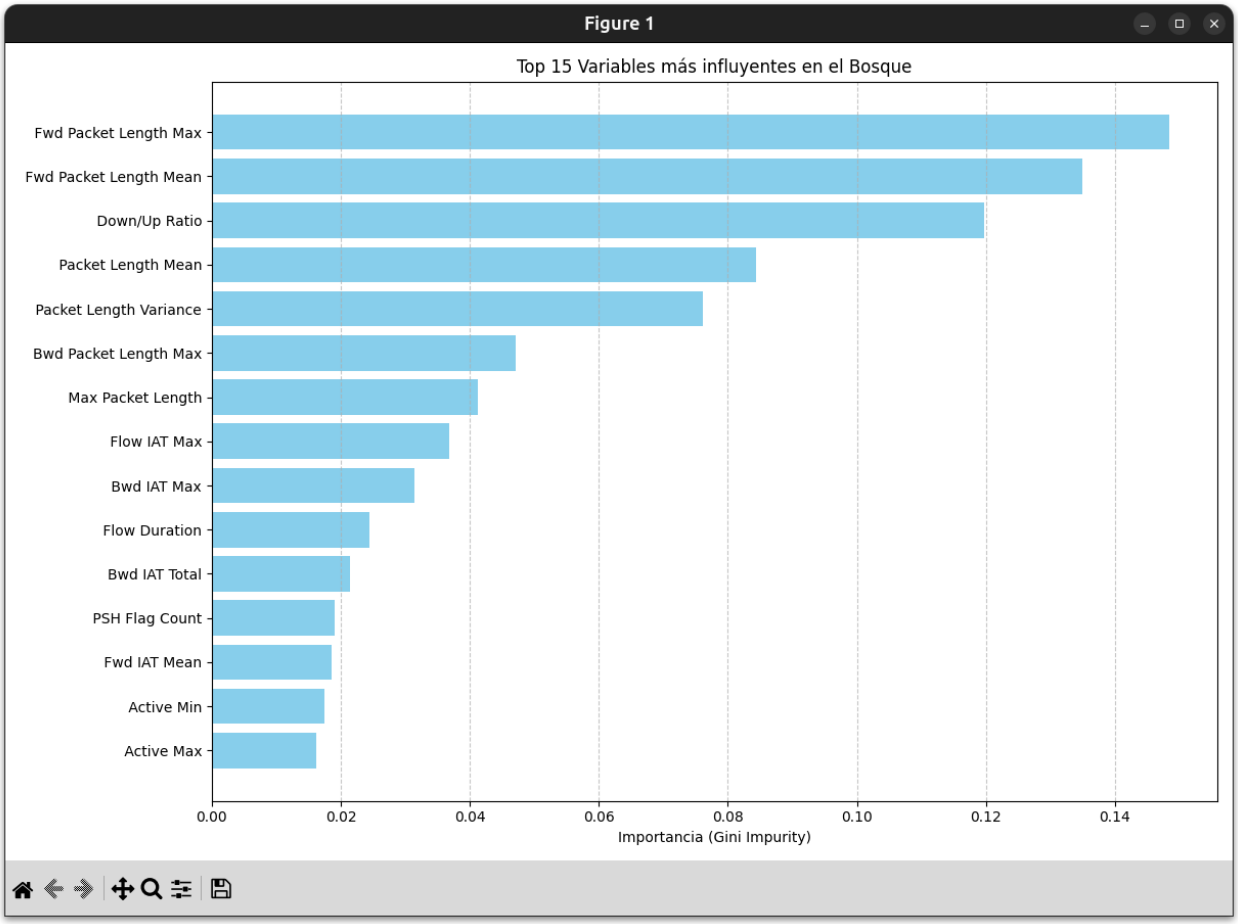

Figura 19: Importancia de Gini sin procesamiento previo: Gráfico de barras

Sin embargo este gráfico únicamente nos muestra las 15 variables con mayor importancia en el dataset. Para ver de cuales se va a prescindir, nos vemos en la necesidad de mostrar todas las varables al completo:

	Variable	Importancia
4	Fwd Packet Length Max	0.148468
6	Fwd Packet Length Mean	0.134929
38	Down/Up Ratio	0.119661
31	Packet Length Mean	0.084369
32	Packet Length Variance	0.076132
7	Bwd Packet Length Max	0.047171
30	Max Packet Length	0.041211
13	Flow IAT Max	0.036812
21	Bwd IAT Max	0.031444
1	Flow Duration	0.024430
18	Bwd IAT Total	0.021443
35	PSH Flag Count	0.019051
15	Fwd IAT Mean	0.018586
43	Active Min	0.017495
42	Active Max	0.016216
9	Flow Bytes/s	0.015946
3	Total Length of Fwd Packets	0.014033
24	Bwd PSH Flags	0.012916
10	Flow Packets/s	0.012358
23	Fwd PSH Flags	0.011146
2	Total Fwd Packets	0.011080
40	Active Mean	0.009539

	Variable	Importancia
16	Fwd IAT Std	0.008866
12	Flow IAT Std	0.008483
11	Flow IAT Mean	0.007695
27	Fwd Packets/s	0.007598
28	Bwd Packets/s	0.006999
29	Min Packet Length	0.006689
39	act_data_pkt_fwd	0.006281
19	Bwd IAT Mean	0.005698
8	Bwd Packet Length Min	0.005276
5	Fwd Packet Length Min	0.005183
20	Bwd IAT Std	0.003690
33	FIN Flag Count	0.003093
34	RST Flag Count	0.000013
0	Destination Port	0.000000
14	Flow IAT Min	0.000000
25	Fwd URG Flags	0.000000
26	Bwd URG Flags	0.000000
22	Bwd IAT Min	0.000000
17	Fwd IAT Min	0.000000
36	ACK Flag Count	0.000000
37	URG Flag Count	0.000000
41	Active Std	0.000000
44	Idle Std	0.000000

Figura 20: Importancia de Gini sin procesamiento previo: Dataframe

En esta imagen se aprecia, a través de un dataframe previo que garantiza una mejor visualización de los datos, aquellos campos que carecen de significado para el modelo. Entre estos campos destacan algunos como *Flow Duration* y tiempos de llegada entre paquetes (IAT) los cuales dependen en mayor medida de la velocidad del entorno de red donde se realicen las pruebas, otras métricas de rendimiento del hardware y de la carga de red en un momento dado (*Flow Byte/s, Flow Packet/s*), y otros parámetros temporales que no interfieren en la naturaleza maliciosa del ataque

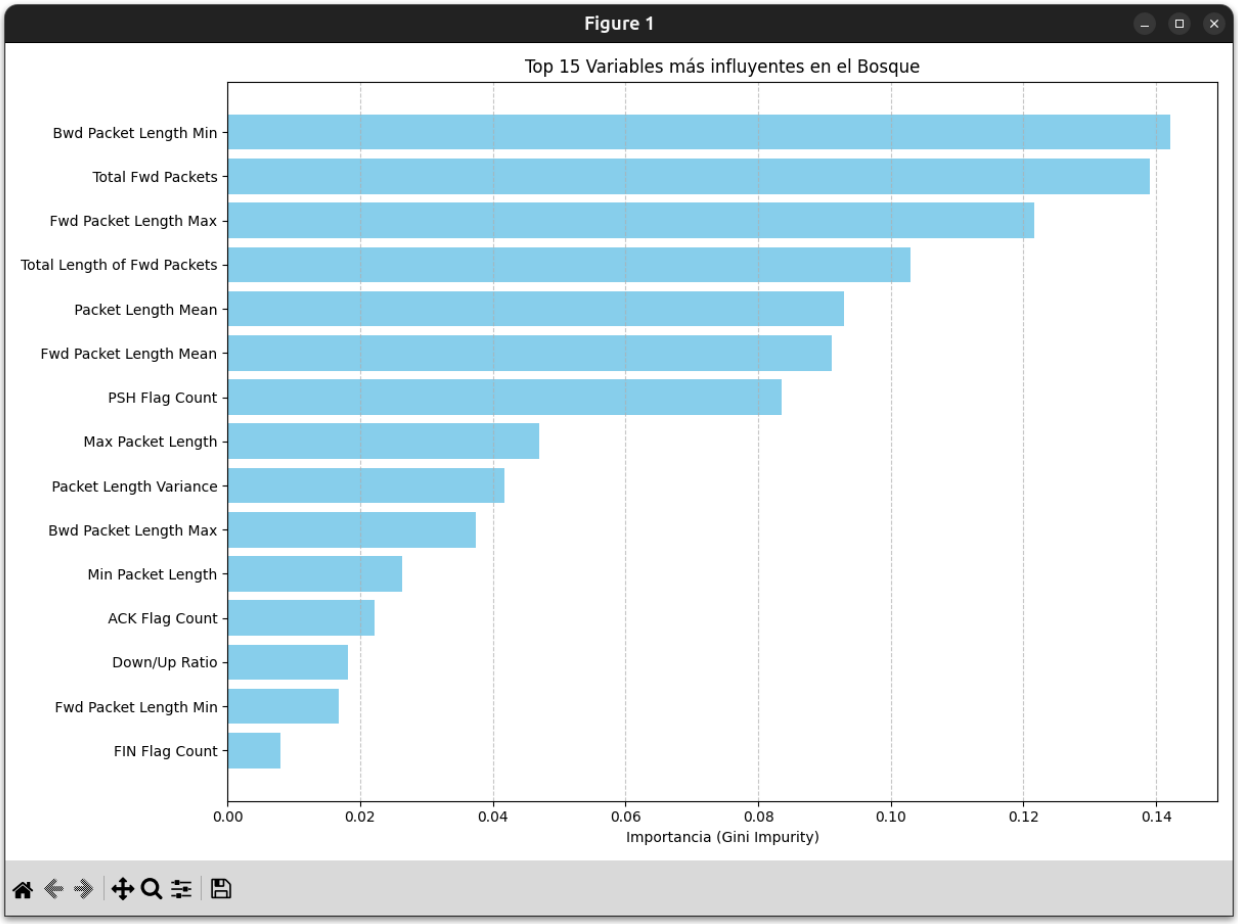

Figura 21: Importancia de Gini con datos procesados: Gráfico de barras

	Variable	Importancia
6	Bwd Packet Length Min	0.142122
0	Total Fwd Packets	0.139046
2	Fwd Packet Length Max	0.121698
1	Total Length of Fwd Packets	0.103031
18	Packet Length Mean	0.092893
4	Fwd Packet Length Mean	0.091092
13	PSH Flag Count	0.083600
17	Max Packet Length	0.046939
19	Packet Length Variance	0.041688
5	Bwd Packet Length Max	0.037425

	Variable	Importancia
16	Min Packet Length	0.026267
14	ACK Flag Count	0.022241
20	Down/Up Ratio	0.018181
3	Fwd Packet Length Min	0.016764
11	FIN Flag Count	0.008017
7	Fwd PSH Flags	0.005094
15	URG Flag Count	0.003903
12	RST Flag Count	0.000000
10	Bwd URG Flags	0.000000
9	Fwd URG Flags	0.000000
8	Bwd PSH Flags	0.000000

Figura 22: Importancia de Gini con datos procesados: Dataframe

Por ello, tras su correcta eliminación, se comprueba que las variables con mayor importancia permanecen prácticamente igual, lo que justifica perfectamente la eliminación de las variables anteriores, obteniendo un modelo mucho más ligero y preparado para analizar características intrínsecas propias de tráfico malicioso.

4.7.2. Primeros resultados

Tras un primer entrenamiento del *Random Forest*, lo resultados no coincidieron con lo esperado, y esto se debe a la naturaleza de los datos de partida. Tras un previo análisis, se identificó un fuerte sesgo en la distribución de muestras de carácter malicioso, ya que ciertos ataques, *DoS Hulk* y *DoS Goldeneye* para ser más específicos, representan una gran descompensación con el resto de amenazas:

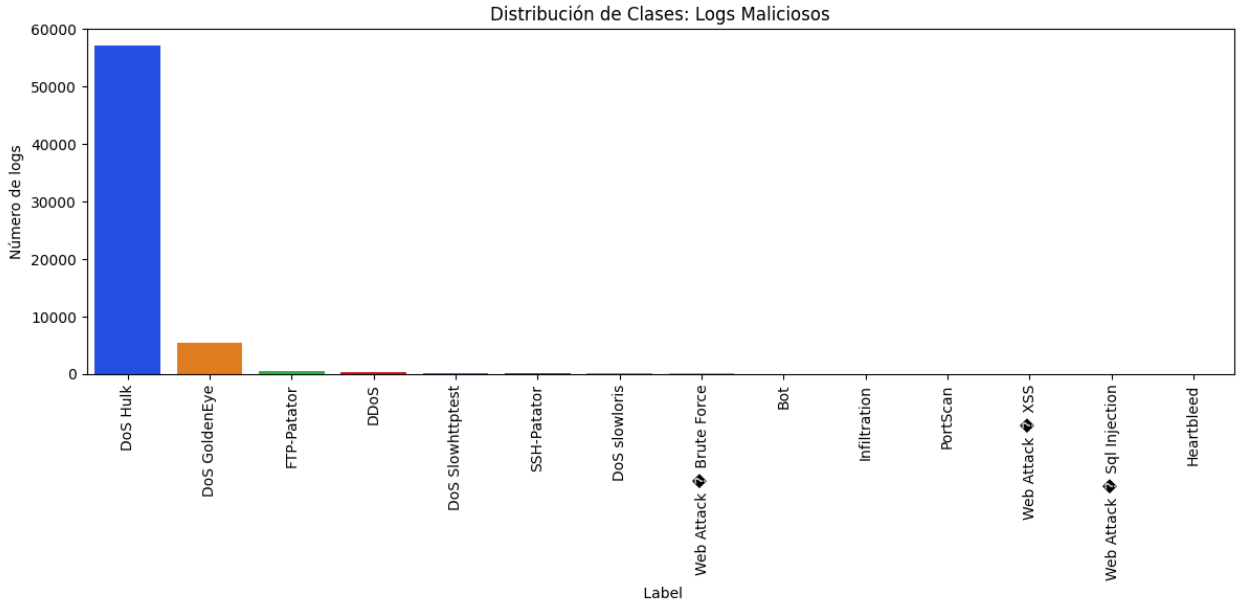

Figura 23: Distribución de clases de carácter malicioso tras análisis de duplicados

Este gráfico muestra el sesgo que se ha hablado anteriormente, mostrando que tras un análisis de datos duplicados la cantidad de datos maliciosos se concentra mucho más entorno a *DoS Hulk* y *DoS Goldeneye*, lo que hace que el resto de opciones sean de carácter invisible en la clasificación al no presentar apenas datos. Este desequilibrio conlleva una serie de problemas en la clasificación del modelo:

- **Sesgo Algorítmico:** Al presentar una desproporción tan grande de datos en las muestras de *Hulk* y *GoldenEye*, los distintos clasificadores del bosque aprenden con mayor detalle este tipo de tráfico pero quedan estancados en un aprendizaje mucho más superficial y general con el resto de ataques.

- **Predominancia de Falsos Positivos:** De manera ligada al punto anterior, al conocer mucho mejor las características de un par de clasificaciones en concreto, el modelo genera una fuerte tendencia a clasificar las opciones que mejor conoce, las cuales son *DoS Hulk* y *DoS Goldeneye*, por una mera cuestión de probabilidad derivada en el entrenamiento.

- **Generalización Deficiente:** Al presentar una gran escasez de muestras en ataques como *SQL Injection* o *Botnets* impiden que el algoritmo identifique patrones muy característicos, generando muy poca confianza en la predicción del resto de clases.

Tras esto se puede explicar el resultado de innumerables pruebas erróneas generadas por el modelo, reproduciendo ataques con distintas herramientas presentadas en (4.5) y observando que los flujos de red generados son etiquetados incorrectamente como *Dos GoldenEye* o *DoS Hulk* en la gran mayoría de los casos. Además, sumado a una baja confianza del modelo a la hora de la clasificación final (normalmente cercana al 40 %) quiere decir que el modelo se encuentra con mucha indecisión debido a este fuerte sesgo.

```
Flujo completado 192.168.1.168:58186 -> 192.168.1.168:80
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
2.0539
Ataque detectado! MSE:  2.0539
 -> RandomForestClassifier: DoS Hulk, (44.77%)
```

```
Flujo completado 192.168.1.168:56012 -> 192.168.1.168:80
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
2.4846
Ataque detectado! MSE:  2.4846
 -> RandomForestClassifier: DoS GoldenEye, (42.58%)
```

Figura 24: Predicciones erróneas en la clasificación inicial

Con esto vemos que las predicciones no son correctas, por lo que se ha de buscar una solución para corregirlas. En la siguiente sección se muestra la medida adoptada para solucionar este problema, mediante la generación de tráfico adicional, implementado directamente en un entorno de red propio y mucho más controlado, con total transparencia a la hora de mostrar las herramientas y metodologías implementadas para mitigar este problema y mejorar la equidad en la detección de todas las opciones posibles.

4.7.3. Captura de datos personalizada

Una vez se ha detectado el sesgo visto en la sección anterior (4.7.2), se ha capturado el tráfico malicioso de la reproducción de ataques en una red doméstica, simulando un entorno de laboratorio controlado para la captura. Para la simulación de ataques de denegación de servicio *DoS* se ha utilizado la herramienta `hping3`, la cual permite generar fuertes inundaciones de paquetes TCP/IP, al igual que `sqlmap` para inyecciones *SQL*, ya que automatiza la detección de vulnerabildades *SQL*.

Por otra parte, nos hemos apoyado en la herramienta de código abierto creada por Janse de Visser y Jan Seidl (7) para ataques *DoS Goldeneye*, la cual se puede ejecutar a través de un fichero de Python, de la misma manera que uno de los scripts creados para este proyecto, `bruteforce.py`, el cual se puede encontrar en el repositorio de este trabajo (6), generando un ataque de fuerza bruta (*Bruteforce*), al igual que se ha hecho con `hydra` como se vio anteriormente en (17).

Por último, ya que no se han implementado todos los ataques que se disponen en los datos originales, se ha concluido con `slowhttptest`, especializándose en ataques de bajo ancho de banda como *DoS Slowhttptest*. Con estos datos se puede entrenar correctamente un modelo

mucho más homogéneo en cuanto a distribución de clases, propiciando un buen desempeño del modelo con predicciones mucho más precisas, ya que el flujo generado para el entrenamiento tiene características mucho más similares a los datos del entorno de pruebas, erradicando el sesgo que provocaba fuertes confusiones en la clasificación final:

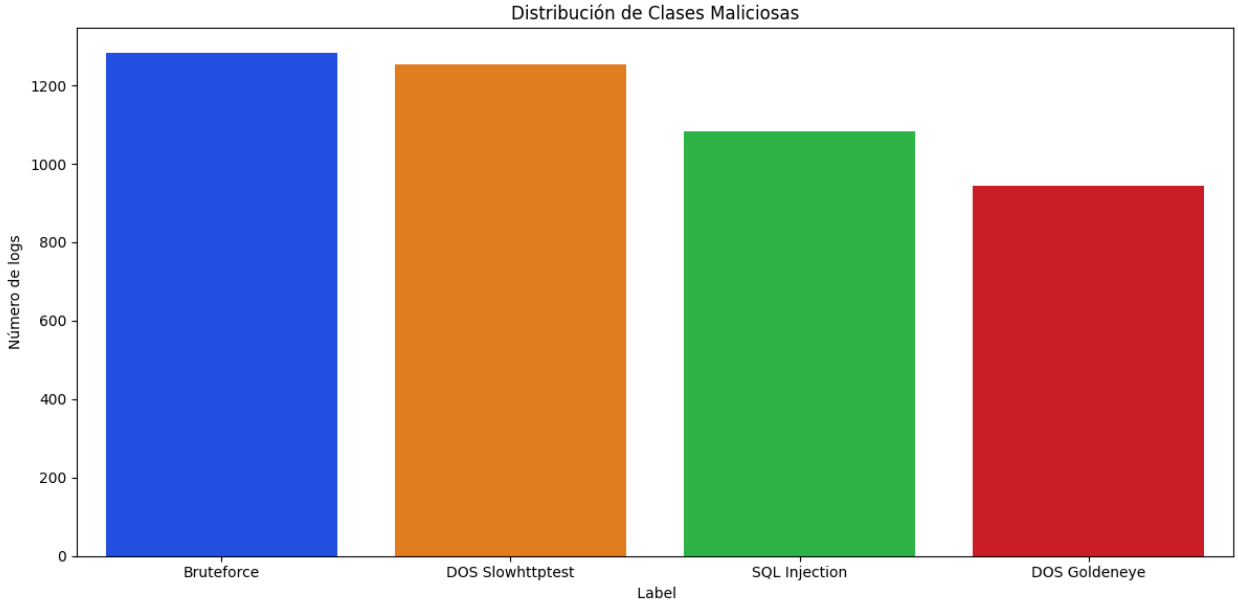

Figura 25: Distribución de clases de carácter malicioso del tráfico capturado

Finalmene, esta captura personalizada de datos resultó ser fundamental para el equilibrio de los datos, ya que permite que el clasificador *Random Forest* sea entrenado con una representación más equitativa y realista de cada categoría de ataque, lo que se traduce en unos resultados mucho más acorde a lo esperado, como se verá en la siguiente sección.

4.7.4. Resultados finales

Antes de mostrar las principales métricas como la `precision`, `accuracy`, `recall` o `score f1`, se van a analizar de manera detallada las muestras generadas ante el desequilibrio de datos que se vio en 4.7.2. Vamos a tomar como ejemplo las inyecciones SQL, tomadas a través de la herramienta `sqlmap`:

	Total Fwd Packets	Total Length of Fwd Packets	Fwd Packet Length Max	Fwd Packet Length Min	Fwd Packet Length Mean	Bwd Packet Length Max	Bwd Packet Length Min	Fwd PSH Flags	Bwd PSH Flags	Fwd URG Flags	Bwd URG Flags	FIN Flag Count	RST Flag Count	PSH Flag Count	ACK Flag Count	URG Flag Count	Min Packet Length	Max Packet Length	Packet Length Mean	Packet Length Variance	Down/Up Ratio	Label
0	5.0	629.0	349.0	68.0	125.8	1370.0	68.0	1.0	1.0	0.0	0.0	1.0	0.0	1.0	1.0	0.0	68.0	1370.0	227.9	168734.766667	1.0	SQL Injection
1	5.0	933.0	653.0	68.0	186.6	1476.0	68.0	1.0	1.0	0.0	0.0	1.0	0.0	1.0	1.0	0.0	68.0	1476.0	268.9	213467.655556	1.0	SQL Injection
2	5.0	901.0	621.0	68.0	180.2	1448.0	68.0	1.0	1.0	0.0	0.0	1.0	0.0	1.0	1.0	0.0	68.0	1448.0	262.9	203386.322222	1.0	SQL Injection
3	5.0	617.0	337.0	68.0	123.4	1360.0	68.0	1.0	1.0	0.0	0.0	1.0	0.0	1.0	1.0	0.0	68.0	1360.0	225.7	165895.566667	1.0	SQL Injection
4	5.0	927.0	647.0	68.0	185.4	1457.0	68.0	1.0	1.0	0.0	0.0	1.0	0.0	1.0	1.0	0.0	68.0	1457.0	266.4	207896.044444	1.0	SQL Injection

Figura 26: Muestras de datos capturados: SQL Injection

Esto revela una homogeneidad muy alta en los datos tomados, ya que se puede identificar muy fácilmente la presencia de patrones que determinan la estructura de este tipo de ataque en concreto. En particular, se pueden encontrar los siguientes patrones en flujos generados por la herramienta `sqlmap`:

- **Simetría de Flujo:** El ratio de subida/bajada (`Down/Up Ratio`) almacena un valor constante de 1.0, al igual que la cantidad total de paquetes enviados (`Total Fwd Packets`) que también presenta el mismo valor, en este caso 5 paquetes por flujo.

- **Flags TCP:** También se puede observar que las banderas de control TCP (`FIN`, `PSH`, `ACK`) presentan el mismo patrón, activándose exactamente las mismas banderas en cada flujo de red generado.

- **Límites de Carga:** Por último se aprecia que las longitudes mínimas de los paquetes (`Min Packet Length`, `Fwd Packet Length Min`, `Bwd Packet Length Min`) presentan un valor homogéneo y constante en todos los flujos capturados, estabilizándose en un total de 68 bytes.

Esto provoca que, durante la fase de evaluación del modelo, se obtengan métricas casi perfectas, alcanzando valores de 0,99 sobre 1 en *Accuracy*, *Precision*, *Recall* y *F1-Score* como consecuencia de la alta homogeneidad de los datos capturados:

```
[*] Entrenando Random Forest...

========================================
 RESULTADOS: RandomForestClassifier
========================================
Accuracy : 0.9956
Precision: 0.9957
Recall   : 0.9956
F1-Score : 0.9956
```

Figura 27: Metricas del entrenamiento del modelo final: RandomForest

Desde una perspectiva analítica, esto justifica matemáticamente las métricas que se han obtenido. Sin embargo, esto salta las alarmas, ya que es un claro ejemplo de **sobreentrenamiento (overfitting)**, ante la grave dificultad para detectar un ataque de forma genérica, teniendo el modelo únicamente la capacidad de detectar la huella de red generada por ciertas herramientas, como `sqlmap`, bajo una configuración en específico en el entorno de pruebas. Si se hace la prueba, se puede verificar que se tiene mucha confianza en las predicciones, como resultado de los fenómenos recientemente explicados:

```
Flujo completado 192.168.1.227:56050 -> 192.168.1.227:80
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 73ms/step
2.0742
Ataque detectado! MSE:  2.0742
 -> RandomForestClassifier: SQL Injection, (86.67%)

Flujo completado 192.168.1.227:56084 -> 192.168.1.227:80
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 52ms/step
2.0684
Ataque detectado! MSE:  2.0684
 -> RandomForestClassifier: SQL Injection, (87.50%)

Flujo completado 192.168.1.227:56124 -> 192.168.1.227:80
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step
2.0854
Ataque detectado! MSE:  2.0854
 -> RandomForestClassifier: SQL Injection, (87.17%)
```

Figura 28: Prediciendo ataque malicioso creado con SQLMap

En conclusión, aunque esto pueda aparentar inicialmente un gran éxito del clasificador, el

análisis pormenorizado de los flujos capturados sugiere una realidad más compleja. La presencia de patrones deterministas (firmas de red como las flags y métricas de volumen y longitud de datos) indica que el modelo converge a una solución muy óptima para nuestro entorno controlado, pero a la vez es potencialmente frágil ante un escenario real. Este fenómeno recalca la importancia de la diversidad en los datos, ya que se demuestra una precisión excepcional para identificar ataques producidos por herramientas en concreto, con el coste de perder la capacidad de generalizar mucho más allá y lograr adaptarse a la enorme variabilidad intrínseca del tráfico malicioso.

4.8. Implementación de Notificaciones y Respuesta ante Incidentes vía Telegram API

A continuación se ha decidido implementar la API de Telegram en este proyecto, lo que proporciona un sistema de respuesta y monitarización remota a través de un bot. Esta implementación permite establecer un canal de comunicación entre la detección del sistema y su propio administrador, lo que facilita la toma de decisiones frente a posibles intrusiones en tiempo real.

La notificación se produce inmediatamente después de la clasificación del *Random Forest*, en caso de que se haya detectado tráfico malicioso, en caso contrario, no se envía ningún mensaje. Esta comunicación se establece mediante peticiones HTTP POST al *endpoint* de la API de Telegram, y es muy importante recalcar que la comunicación es bidereccional, de manera que al detectarse un posible ataque el administrador puede decidir si bloquear la dirección IP o no, como se puede ver a continuación:

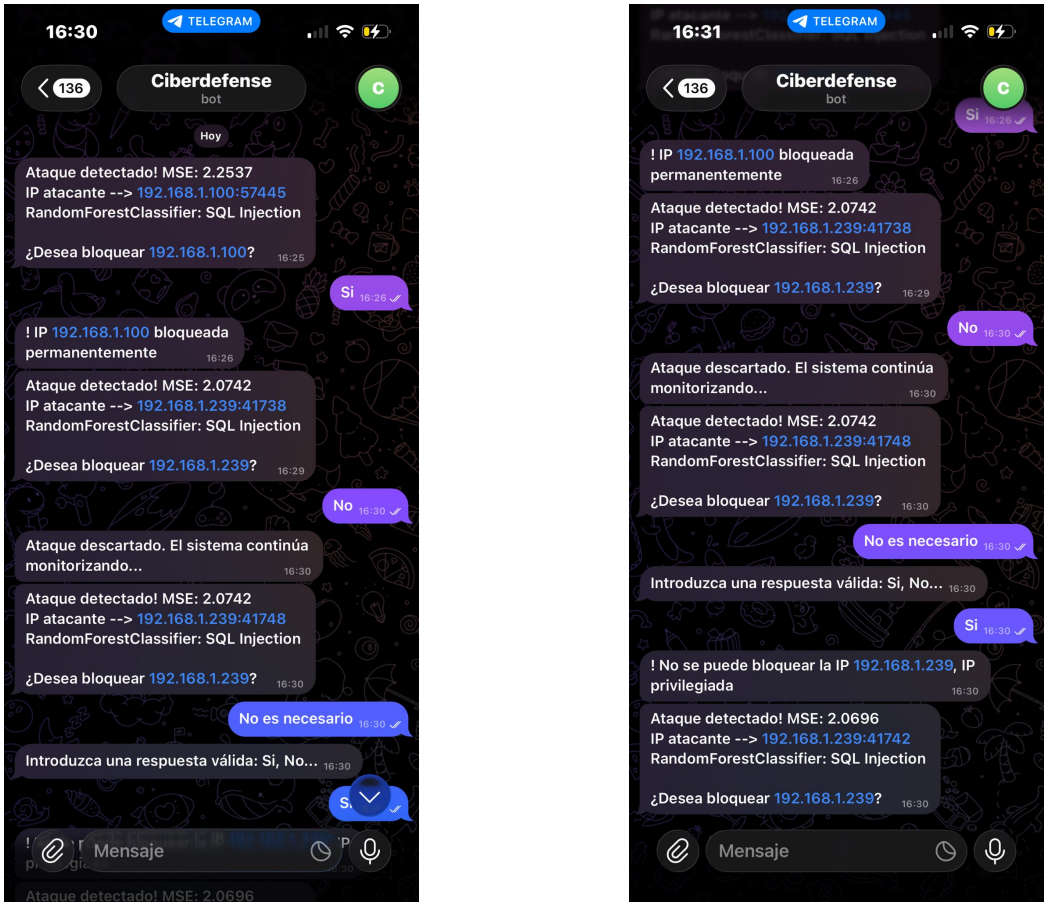

Figura 29: Implementación con la API de Telegram

Con la imagen anterior se puede apreciar el flujo del sistema. Se notifica al administrador, de manera que se puede proceder con el bloqueo de la IP atacante, y en caso afirmativo, se hace una comprobación previa para no bloquear ciertas IPs críticas para el servidor. En caso contrario, se continúa con la detección de anomalías. Esta es la cúlpide del sistema, ya que permite notificar al administrador del servidor de un posible ataque, ofreciendo las medidas necesarias para hacer frente a este tipo de amenazas y mitigando los daños que puedan causar.

CAPÍTULO

5

conclusiones

Bibliografía

[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.

[2] Canadian Institute for Cybersecurity, "Cicids2017 dataset," 2017. Intrusion detection evaluation dataset including benign and attack traffic captures.

[3] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 108–116, SCITEPRESS, 2018.

[4] S. learn Developers, "Scikit-learn: Machine learning in python." https://scikit-learn.org, 2026. Documentación oficial.

[5] hping project, "Documentación de hping3," 2026.

[6] R. N. González, "Repositorio del trabajo de fin de grado." https://github.com/rnoguer22/AnomaliasTFG.git, 2026. Código y recursos utilizados en el desarrollo del TFG.

[7] J. de Visser and J. Seidl, "Goldeneye: A http dos test tool." https://github.com/jseidl/GoldenEye, 2021.

[8] C. C. Aggarwal, *Outlier Analysis*. Springer, 2 ed., 2017.

[9] R. Jain, *The Art of Computer Systems Performance Analysis*. John Wiley & Sons, 1991.

[10] hping project, "Guía de inicio de hping3," 2026.

[11] Apache Software Foundation, "Documentación oficial de apache http server," 2026.

[12] Z. Aouini, "Nfstream: A flexible network data analysis framework," 2026. Framework para el análisis de flujos de red mediante Deep Packet Inspection (DPI).

[13] B. D. A. G. and M. Stampar, "sqlmap: Herramienta automática para inyección sql y explotación de bases de datos," 2026.

[14] M. Heuse, "Thc-hydra: Un cracker de logueo de red muy rápido," 2026. Herramienta de auditoría para ataques de fuerza bruta en múltiples protocolos.

[15] J. An and S. Cho, "Variational autoencoder based anomaly detection using reconstruction probability," *Special Lecture on IE*, vol. 2, no. 1, pp. 1–18, 2015.

[16] D. Kwon, H. Kim, J. Kim, S. C. Suh, I. Kim, and J. H. Kim, "A survey of deep learning-based network intrusion detection: Every bit counts," *IEEE Access*, vol. 7, 2019.